

## 第3章 计算机系统

计算机系统是由硬件系统和软件系统组成的，通过建立系统软件和应用软件来提供解决现实问题的方法。本章简要介绍计算机系统的结构、存储器系统、输入输出系统、指令系统、多处理机系统、操作系统等基础知识。

### 3.1 计算机系统概述

#### 3.1.1 计算机系统层次结构

从不同角度分析，计算机系统有不同的划分层次，但通常可划分为包括硬件层、系统层和应用层的多级层次结构，如图 3-1 所示。

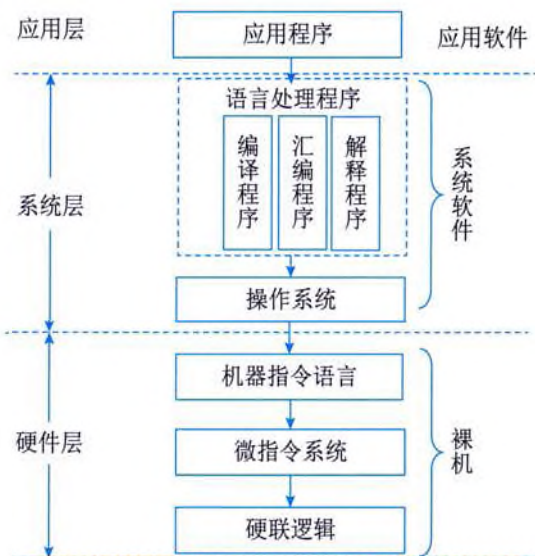


图 3-1 计算机系统的多级层次结构

(1) 硬件层（裸机）。该层可细分为硬联逻辑级、微程序级和传统机器级。其中，硬联逻辑级是计算机的内核，由门电路、触发器等逻辑电路组成；微程序级的机器语言是微指令集；传统机器级的机器语言是该机的指令集。

(2) 系统层。该层可细分为操作系统级、语言处理程序级（如编译程序、汇编程序和解释程序）。其中，操作系统级要管理计算机系统软硬件资源；语言处理程序级的作用是将高级语言或汇编语言编写的程序翻译成某种机器语言程序，使程序可在计算机上运行。

(3) 应用层。该层是针对不同用户而设计的各种应用程序，即面向实际用户的各种应用软件。

### 3.1.2 计算机系统硬件

计算机自 1946 年诞生以来，虽然其制造技术已经发生了巨大的变化，但就其体系而言，都是基于存储程序的原理，也称为冯·诺依曼体系结构。计算机的硬件由 5 个基本部分组成：运算器、控制器、存储器、输入设备和输出设备，如图 3-2 所示。图中实线箭头“ $\longrightarrow$ ”代表数据或指令，在计算机内部表现为二进制数；虚线箭头“ $\cdots\cdots\longrightarrow$ ”代表控制信号，在计算机内部起控制作用。

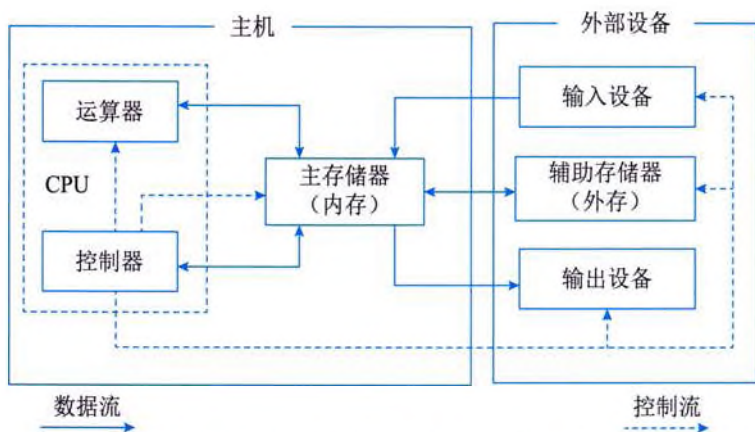


图 3-2 计算机的冯·诺依曼体系结构

运算器、控制器等部件被集成在一起，统称为中央处理单元（Central Processing Unit, CPU）。CPU 是硬件系统的核心，用于数据的加工处理，能完成各种算术、逻辑运算及控制功能。存储器是计算机系统中的记忆设备，分为内部存储器和外部存储器。前者速度快、容量小，一般用于临时存放程序、数据及中间结果。而后者容量大、速度慢，可以长期保存程序和数据。输入设备和输出设备合称为外部设备（简称外设），输入设备用于输入原始数据及各种命令，而输出设备则用于输出计算机运行的结果。

### 3.1.3 计算机系统软件

按照功能，计算机系统软件可以分为系统软件和应用软件两大类。

（1）系统软件：用于实现计算机系统的管理、调度、监视和服务等功能。通常将系统软件分为 5 类：操作系统、语言处理程序、服务性程序、数据库管理系统和计算机网络软件。

（2）应用软件：是为解决某种应用问题而编制的一些程序。应用软件是用户或第三方软件公司为各自业务开发和使用的各种软件，种类繁多。例如，财务管理软件、项目管理软件、营销管理软件等。

另外，还应该注意“固件”（firmware）的概念，它是指那些存储在能永久保存信息的器件（例如，EPROM 或 EEPROM）中的程序，是具有软件功能的硬件。通过固件，操作系统才能按标准的设备驱动程序实现特定机器的运转。固件的性能指标介于硬件与软件之间，其吸收了软、硬件各自的优点，执行速度快于软件，灵活性优于硬件。

## 3.2 存储器系统

存储器是用来存放程序和数据部件,它是一个记忆装置,也是计算机能够实现“存储程序控制”的基础。在计算机系统中,规模较大的存储器往往分成若干级,称为存储器系统。传统的存储器系统一般分为高速缓冲存储器(Cache)、主存储器、辅助存储器三级。存储器中数据常用的存取方式有4种,分别是顺序存取、直接存取、随机存取和相联存取。

存储器系统主要有4个性能指标,分别是存取时间、存储器带宽、存储器周期和数据传输率。

### 3.2.1 主存储器

主存储器(Main Memory)简称主存或内存,是计算机硬件的重要部件,其作用是存放计算机运行时的程序和数据,并能由CPU直接随机地进行读/写。由于CPU要频繁地访问主存,所以主存的性能在很大程度上影响了整个计算机系统的性能。根据工艺和技术不同,主存可分为随机存取存储器(Random Access Memory, RAM)和只读存储器(Read Only Memory, ROM)。

(1) 随机存取存储器既可以写入也可以读出,但断电后信息无法保存,因此只能用于暂存数据。RAM又可分为DRAM(Dynamic RAM, 动态RAM)和SRAM(Static RAM, 静态RAM)两种。DRAM的信息会随时间逐渐消失,因此需要定时对其进行刷新,以维持信息不丢失;SRAM在不断电的情况下,信息能够一直保持而不会丢失信息。

(2) 只读存储器的内容只能随机读出而不能写入。其特点是信息一旦写入ROM,即使断电,写入的内容也不会丢失。ROM一般用于存放系统程序BIOS(Basic Input Output System, 基本输入输出系统)、专用的子程序,或用作函数发生器、字符发生器及微程序控制器中的控制存储器。

### 3.2.2 辅助存储器

辅助存储器简称辅存,用于存放需持久性存储的信息。其特点是存储器容量大、可靠性高、价格低。常用的辅存有磁带存储器、硬盘存储器、磁盘阵列和光盘存储器等。

#### 1. 磁带存储器

磁带存储器是由磁带机及其控制器组成的,是一种顺序存取的设备。磁带控制器是连接计算机与磁带机之间的接口设备,是计算机在磁带上存取数据用的控制电路设备。磁带的优点是存储容量大、便于携带、价格便宜,缺点是存取速度慢。

#### 2. 硬盘存储器

硬盘存储器简称硬盘,属于直接存取设备,即存取磁盘上任一物理块的时间不依赖于该物理块所处的位置。硬盘分为三类:机械硬盘(HDD)、固态硬盘(SSD)和混合硬盘(SSHD)。

(1) 机械硬盘是传统硬盘,由磁性盘片、驱动系统、读写系统和控制系统组成,如图3-3所示。其信息分布呈以下层次:记录面、圆柱面、磁道和扇区。也就是其存取地址可表示为:

驱动器号、柱面号、磁头(读写头)号、扇区号。

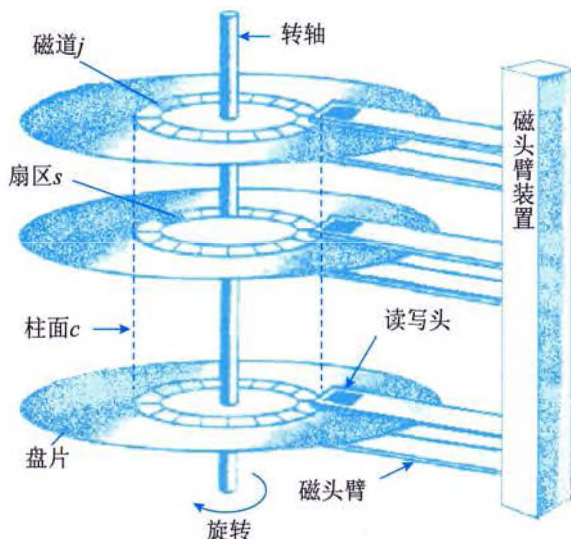


图 3-3 机械硬盘的结构

通常硬盘标称的容量是指格式化容量，即用户实际可以使用的存储容量，而非格式化容量是指磁记录介质上全部的磁化单元数。格式化容量一般约为非格式化容量的 60% ~ 70%。格式化存储容量的计算公式是：

$$\text{存储容量} = n \times t \times s \times b$$

其中： $n$  为保存数据的总记录面数， $t$  为每面的磁道数， $s$  为每道的扇区数， $b$  为每个扇区存储的字节数。

记录密度是指硬盘存储器上单位长度或单位面积所存储的二进制信息量，通常以道密度和位密度表示。道密度是指沿半径方向上单位长度的磁道数目，位密度是指沿磁道方向上单位长度所记录的二进制信息的位数。

硬盘的存取时间主要包括三个部分：第一部分是指磁头从原先位置移动到目的磁道所需要的时间，称为寻道时间或查找时间；第二部分是指到达目的磁道以后，等待被访问的记录块旋转到磁头下方的等待时间；第三部分是指信息的读写操作时间。

由于寻找不同磁道和等待不同记录块所花的时间不同，所以通常取它们的平均值。因为读/写操作时间比较快，相对于平均寻道时间  $T_s$  和平均等待时间  $T_w$  来说可以忽略不计。所以，磁盘的平均存取时间  $T_a$  为：

$$T_a \approx T_s + T_w = \frac{t_{smin} + t_{smax}}{2} + \frac{t_{wmin} + t_{wmax}}{2}$$

设置硬盘缓存是为了解决硬盘内部与接口之间速度不匹配的问题，从而进一步提高硬盘的读/写速度。

(2) 固态硬盘是由多个闪存芯片加主控以及缓存组成的阵列式存储设备，属于以固态电子存储芯片阵列制成的硬盘。SSD 与 HDD 相比，读取速度快，寻道时间可忽略不计，但其价格更高。

(3) 混合硬盘是机械硬盘与固态硬盘的结合体, 采用容量较小的闪存用来存储常用文件, 而磁盘才是最主要的存储介质, 闪存仅起到缓冲作用, 将更多的常用文件保存到闪存中, 可以减小寻道时间, 进一步提高读/写操作效率。

### 3. 磁盘阵列

廉价磁盘冗余阵列 (Redundant Array of Inexpensive Disks, RAID) 技术旨在缩小日益扩大的 CPU 速度和磁盘存储器速度之间的差距。其策略是用多个较小的磁盘驱动器替换单一的大容量磁盘驱动器, 同时合理地在多个磁盘上分布存放数据以支持同时从多个磁盘进行读写, 从而改善系统的 I/O 性能。RAID 现在代表独立磁盘冗余阵列 (Redundant Array of Independent Disks), “Independent” (独立) 强调 RAID 技术所带来的性能改善和更高的可靠性。RAID 应用的主要技术有分块技术、交叉技术和重聚技术。

RAID 分为以下 8 个级别:

(1) RAID 0 (无冗余和无校验的数据分块): 具有最高的 I/O 性能和最高的磁盘空间利用率, 易管理, 但系统的故障率高, 属于非冗余系统。

(2) RAID 1 (磁盘镜像阵列): 由磁盘对组成, 每一个工作盘都有其对应的镜像盘, 上面保存着与工作盘完全相同的数据拷贝, 具有最高的安全性, 但磁盘空间利用率只有 50%。RAID 1 主要用于存放系统软件、数据以及其他重要文件。

(3) RAID 2 (采用纠错海明码的磁盘阵列): 采用了海明码纠错技术, 用户需增加校验盘来提供单纠错和双纠错功能。对数据的访问涉及阵列中的每一个盘。大量数据传输时 I/O 性能较高, 但不利于小批量数据传输。实际应用中很少使用。

(4) RAID 3 和 RAID 4 (采用奇偶校验码的磁盘阵列): 把奇偶校验码存放在一个独立的校验盘上。如果有一个盘失效, 其上的数据可以通过对其他盘上的数据进行异或运算得到。读数据很快, 但因为写入数据时要计算校验位, 速度较慢。RAID 3 采用位交叉奇偶校验码, RAID 4 采用块交叉奇偶校验码。

(5) RAID 5 (无独立校验盘的奇偶校验码磁盘阵列): 与 RAID 4 类似, 但没有独立的校验盘, 校验信息分布在组内所有盘上。RAID 4 和 RAID 5 使用了独立存取技术, 阵列中每一个磁盘都相互独立地操作, 故 I/O 请求可以并行处理。

(6) RAID 6 (具有独立的数据硬盘与两个独立的分布式校验方案): 在 RAID 6 的阵列中设置了一个专用的、可快速访问的异步校验盘。该盘具有独立的数据访问通路, 但其性能改进有限, 价格却很昂贵。

(7) RAID 7 (具有最优化的异步高 I/O 速率和高数据传输率的磁盘阵列): 是对 RAID 6 的改进。在这种阵列中的所有磁盘都具有较高的传输速度, 有着优异的性能, 是目前最高档次的磁盘阵列。

(8) RAID 10 (高可靠性与高性能的组合): 由多个 RAID 等级组合而成, 建立在 RAID 0 和 RAID 1 基础上。RAID 1 是一个冗余的备份阵列, 而 RAID 0 是负责数据读写的阵列, 因此又称为 RAID 0+1。由于利用了 RAID 0 极高的读写效率和 RAID 1 较高的数据保护和恢复能力, 使 RAID 10 成为了一种性价比较高的等级。

在以上级别中，需要根据可用性（数据冗余）、性能和成本进行综合选择。

#### 4. 光盘存储器

光盘存储器是利用激光束在记录表面存储信息，根据激光束的反射光来读出信息。根据性能和用途的不同，光盘存储器可分为4种类型，分别为CD-ROM、CD-R、CD-RW和DVD-ROM。

(1) CD-ROM (Compact Disc Read-Only Memory, 只读压缩盘) 又称固定型光盘。它由生产厂家预先写入数据和程序，使用时用户只能读出，不能修改或写入新内容。CD-ROM 的读取目前有三种方式，分别是恒定角速度、恒定线速度和部分恒定角速度。

(2) CD-R 光盘采用 WORM (Write One Read Many) 标准，光盘可由用户写入信息，写入后可以多次读出；但只能写入一次，信息写入后将不能再修改。

(3) CD-RW 光盘是可以写入、擦除、重写的可逆性记录系统，这种光盘类似于磁盘，可重复读写。

(4) DVD-ROM 技术类似于 CD-ROM 技术，但是可以提供更高的存储容量。DVD 可以分为单面单层、单面双层、双面单层和双面双层4种物理结构。

### 3.2.3 高速缓冲存储器

Cache 的功能是提高 CPU 数据输入输出的速率，突破所谓的“冯·诺依曼瓶颈”，即 CPU 与存储系统间的数据传送带宽限制。高速存储器能以极高的速率进行数据的访问，但其价格高昂，通常在 CPU 和内存之间设置小容量的 Cache，利用 Cache 速度快的特点，通过优化调度算法，使系统的性能得到改善。

#### 1. Cache 的基本原理

使用 Cache 改善系统性能的依据是程序的局部性原理。程序访问的局部性有两个方面的含义，分别是时间局部性和空间局部性。其中，时间局部性是指如果一个存储单元被访问，则可能该单元会很快被再次访问；空间局部性是指如果一个存储单元被访问，则该单元邻近的单元也可能很快被访问。

根据程序的局部性原理，可以把目前常用或将要用到的信息预先放在 Cache 中。当 CPU 需要读取数据时，首先在 Cache 中查找是否有所需内容，如果有，则直接从 Cache 中读取；若没有，再从内存中读取该数据，然后同时送往 CPU 和 Cache。

如果  $h$  表示对 Cache 的访问命中率（“ $1-h$ ”称为失效率，或者未命中率）， $t_1$  表示 Cache 的周期时间， $t_2$  表示内存的周期时间，以读操作为例，使用“Cache+主存储器”的系统的平均周期为  $t_3$ 。则：

$$t_3 = t_1 \times h + t_2 \times (1-h)$$

系统的平均存储周期与命中率有很密切的关系，命中率的提高即使很小也能导致性能上的较大改善。例如，设某计算机主存的读/写时间为 100ns，有一个指令和数据合一的 Cache，已知该 Cache 的读/写时间为 10ns，取指令的命中率为 98%，取数的命中率为 95%。在执行某类

程序时,约有 1/5 指令需要存/取一个操作数。假设指令流水线在任何时候都不阻塞,则设置 Cache 后,每条指令的平均访存时间约为:

$$(2\% \times 100\text{ns} + 98\% \times 10\text{ns}) + 1/5 \times (5\% \times 100\text{ns} + 95\% \times 10\text{ns}) = 14.7\text{ns}$$

## 2. 映射机制

当 CPU 发出访存请求后,存储器地址先被送到 Cache 控制器以确定所需数据是否已在 Cache 中,若命中,则直接对 Cache 进行访问。这个过程称为 Cache 的地址映射(映像)。在 Cache 的地址映射中,主存和 Cache 将均分成容量相同的块(页)。常见的映射方法有直接映射、全相联映射和组相联映射。

(1) 直接映射。直接映射方式以随机存取存储器作为 Cache,硬件电路较简单。直接映射是一种多对一的映射关系,但一个主存块只能够复制到 Cache 的一个特定位置上去。直接映射的关系可以用下列公式来表示:

$$K = I \bmod C$$

其中,  $K$  为 Cache 的块号,  $I$  为主存的页号,  $C$  为 Cache 的块数。

直接映射方式的优点是容易实现,缺点是不够灵活,有可能使 Cache 的存储空间得不到充分利用。例如,假设 Cache 有 8 块,则主存的第 1 页与第 17 页需同时复制到 Cache 的第 1 块,即使 Cache 其他块空闲,也有一个主存页不能写入 Cache。

(2) 全相联映射。全相联映射使用相联存储器组成 Cache。在全相联映射方式中,主存的每一页可以映射到 Cache 的任一块。如果淘汰 Cache 中某一块的内容,则可调入任一主存页的内容,因而较直接映射方式灵活。

在全相联映射方式中,主存地址不能直接提取 Cache 块号,而是需要将主存页标记与 Cache 各块的标记逐个比较,直到找到标记符合的块(访问 Cache 命中),或者全部比较完后仍无符合的标记(访问 Cache 失败)。该映射方式最大的缺点是速度很慢,失去了高速缓存的作用。如果让主存页标记与各 Cache 标记同时比较,则成本又太高。全相联映射方式因比较器电路难于设计和实现,只适用于小容量的 Cache。

(3) 组相联映射。组相联映射是直接映射和全相联映射的折中方案。它将 Cache 中的块再分成组,通过直接映射的方式决定组号,通过全相联映射的方式决定 Cache 中的块号。在组相联映射方式中,主存中一个组内的页数与 Cache 的分组数相同。

例如,容量为 64 块的 Cache 采用组相联映射方式,每块大小为 128 个字,每 4 块为一组,即 Cache 分为  $64/4=16$  组。若主存容量为 4096 页,且以字编址。首先,根据主存与 Cache 块的容量一致要求,即每个内存页的大小也是 128 个字,因此共有  $128 \times 4096$  个字 ( $2^{19}$  个字),即主存地址需要 19 位。因为 Cache 分为 16 组,所以主存需要分为  $4096/16=256$  组(每组 16 页),即  $2^8$  组,因此主存组号需 8 位。

按照上述划分方法,主存每一组的第 1 页映射到 Cache 的第 1 组,主存每一组的第 2 页映射到 Cache 的第 2 组,以此类推。因为主存中一个组内的页数与 Cache 的分组数相同,所以主存每一组的最后一页映射到 Cache 的最后一组。

需要注意的是,有关组相联映射的划分方法不止一种。例如,另外一种方式是主存不分组,

而是根据下列公式直接进行映射。

$$J = I \bmod Q$$

其中,  $J$  为 Cache 的组号,  $I$  为主存的页号,  $Q$  为 Cache 的组数。

在组相联映射中, 由于 Cache 中每组有若干可供选择的块, 故它在映射定位方面较直接映射方式灵活; 每组块数有限, 付出的代价小, 可以根据设计目标选择组内块数。

### 3. 替换算法

当 Cache 产生了一次访问未命中之后, 相应的数据应同时读入 CPU 和 Cache。但是当 Cache 已存满数据后, 新数据必须替换 (淘汰) Cache 中的某些旧数据。最常用的替换算法有随机算法、先进先出算法和近期最少使用算法三种。

### 4. 写操作

因为需要保证缓存在 Cache 中的数据与内存中的内容一致, 相对读操作而言, Cache 的写操作比较复杂, 常用的有以下几种方法:

(1) 写直达 (Write Through)。当要写 Cache 时, 数据同时写回内存, 有时也称为写通。当某一块需要替换时, 也不必把这一块写回到主存中去, 新调入的块可以立即把这一块覆盖掉。这种方法实现简单, 并能随时保持主存数据的正确性, 但可能会增加多次不必要的主存写入, 从而降低存取速度。

(2) 写回 (Write Back)。CPU 修改 Cache 的某一块后, 相应的数据并不立即写入内存单元, 而是当该块从 Cache 中被淘汰时, 才把数据写回到内存中。在采用这种更新策略的 Cache 块表中, 一般有一个标志位, 当一块中的任何一个单元被修改时, 标志位被置“1”。在需要替换掉这一块时, 如果标志位为 1, 则必须先把这一块写回到主存中去, 之后才能再调入新的块; 如果标志位为 0, 则这一块不必写回主存, 只要用新调入的块覆盖掉这一块即可。这种方法的优点是操作速度快, 缺点是因主存中的字块未随时修改而有可能出错。

(3) 标记法。对 Cache 中的每一个数据设置一个有效位。数据进入 Cache, 有效位置 1; 当 CPU 要对该数据进行修改时, 数据只需写入内存并同时将该有效位置 0。当要从 Cache 中读取数据时需要测试其有效位, 若为“1”, 则直接从 Cache 中取数, 否则从内存中取数。

## 3.2.4 网络存储技术

常用的网络存储技术主要有以下三种: 直接附加存储 (Direct Attached Storage, DAS)、网络附加存储 (Network Attached Storage, NAS) 和存储区域网络 (Storage Area Network, SAN)。

### 1. 直接附加存储

DAS 是将存储设备通过 SCSI (Small Computer System Interface, 小型计算机系统接口) 电缆直接连到服务器, 其本身是硬件的堆叠, 存储操作依赖于服务器, 不带有任何存储操作系统。因此, 有些文献也把 DAS 称为 SAS (Server Attached Storage, 服务器附加存储)。DAS 的适用情形如下:

(1) 服务器在地理分布上很分散, 想要通过 SAN 或 NAS 在它们之间进行互连非常困难。

(2) 存储系统必须被直接连接到应用服务器（例如，Microsoft Cluster Server 或某些数据库使用的“原始分区”）上。

(3) 包括许多数据库应用和应用服务器在内的应用，它们需要直接连接到存储器上。

目前 DAS 基本被 NAS 所代替，主要原因是 DAS 直接将存储设备连接到服务器上，这导致它在传递距离、连接数量、传输速率等方面都受到限制；当存储容量增加时，DAS 方式很难扩展，这对存储容量的升级是一个巨大的瓶颈；由于数据的读取都要通过服务器来处理，必然导致服务器的处理压力增加，数据处理和传输能力将大大降低；此外，当服务器出现宕机等异常时，也会波及存储数据，使其无法使用。

## 2. 网络附加存储

NAS 的存储设备类似于一个专用的文件服务器，它去掉了通用服务器的大多数计算功能，而仅提供文件系统功能，从而降低了设备的成本。NAS 以数据为中心，将存储设备与服务器分离，其存储设备在功能上完全独立于网络中的主服务器，客户机与存储设备之间的数据访问不再需要文件服务器的干预，同时，它允许客户机与存储设备之间进行直接的数据访问，所以不仅响应速度快，而且数据传输速率也很高。

采用 NAS 技术的存储设备不再通过 I/O 总线隶属于某个特定的服务器，而是通过网络接口与网络直接相连，由用户通过网络访问。NAS 的系统结构如图 3-4 所示。

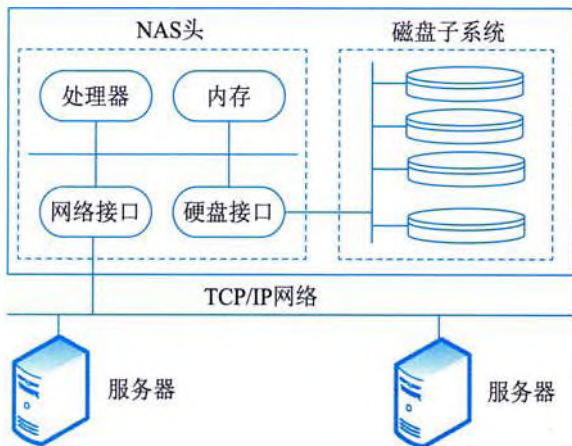


图 3-4 NAS 的系统结构

NAS 技术支持多种 TCP/IP 网络协议，主要是通过网络文件系统（Net File System, NFS）和通用 Internet 文件系统（Common Internet File System, CIFS）来进行文件访问，所以 NAS 的性能特点是进行小文件级的共享存取。在具体使用时，NAS 设备通常配置为文件服务器，通过使用基于 Web 的管理界面来实现系统资源的配置、用户配置管理和用户访问登录等。

NAS 技术支持即插即用，可以在网络的任一位置建立存储。基于 Web 管理，从而使设备的安装、使用和管理更加容易。NAS 可以很经济地解决存储容量不足的问题，但难以获得满意的性能。

### 3. 存储区域网络

SAN 是通过专用交换机将磁盘阵列与服务器连接起来的高速专用子网。它没有采用文件共享存取方式，而是采用块（block）级别存储。SAN 是通过专用高速网将一个或多个网络存储设备和服务器连接起来的专用存储系统，其最大特点是将存储设备从传统的以太网中分离出来，成为独立的存储区域网络。SAN 的系统结构如图 3-5 所示。

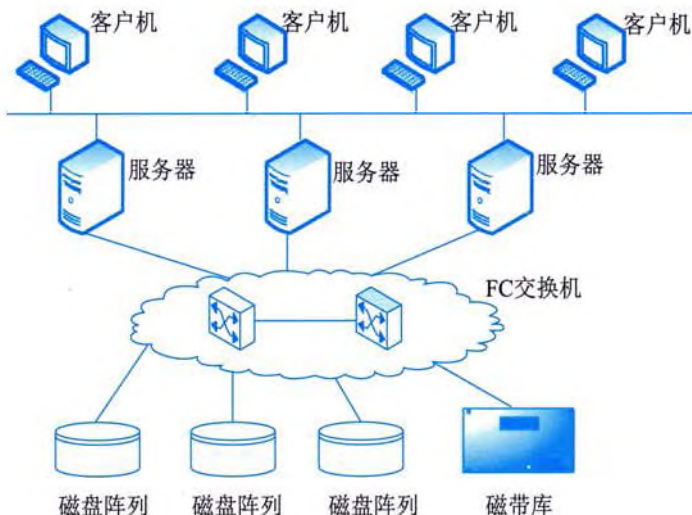


图 3-5 SAN 的系统结构

根据数据传输过程采用的协议，其技术划分为 FC SAN、IP SAN 和 IB SAN。

(1) FC SAN。FC（Fiber Channel，光纤通道）和 SCSI 接口一样，最初也不是为硬盘设计的接口技术，而是专门为网络系统设计的，随着存储系统对速度的需求，才逐渐应用到硬盘系统中。光纤通道的主要特性有：热插拔性、高速带宽、远程连接、连接设备数量大等。它是当今最昂贵和复杂的存储架构，需要在硬件、软件和人员培训方面进行大量投资。

FC SAN 由接口（SCSI、FC）、连接设备（交换机、路由器）和协议（IP、SCSI）三个基本的组件构成。这三个组件加上附加的存储设备和服务器就构成了一个 SAN 系统。它是专用、高速、高可靠的网络，允许独立、动态地增加存储设备，使得管理和集中控制更加简化。

FC SAN 有两个较大的缺陷，分别是成本和复杂性，其原因就是使用了 FC。在光纤通道上部署 SAN，需要每个服务器上有 FC 适配器、专用的 FC 交换机和独立的布线基础架构。这些设施使成本大幅增加，更不用说精通 FC 协议的人员培训成本。

(2) IP SAN。IP SAN 是基于 IP 网络实现数据块级别存储方式的存储网络。由于设备成本低，配置技术简单，可共享和使用大容量的存储空间，因而逐渐获得广泛的应用。

在具体应用上，IP 存储主要是指 iSCSI（internet SCSI）。作为一种新兴的存储技术，iSCSI 基于 IP 网络实现 SAN 架构，既具备了 IP 网络配置和管理简单的优势，又提供了 SAN 架构所拥有的强大功能和扩展性。iSCSI 是连接到一个 TCP/IP 网络的直接寻址的存储库，通过使用 TCP/IP 协议对 SCSI 指令进行封装，可以使指令能够通过 IP 网络进行传输，而过程完全不依赖于地点。

iSCSI 的优势主要表现在三个方面：建立在 SCSI、TCP/IP 这些稳定和熟悉的标准上，因此

安装成本和维护费用都很低；支持一般的以太网交换机而不是特殊的光纤通道交换机，从而减少了异构网络和电缆；通过 IP 传输存储命令，因此可以在整个 Internet 上传输，没有距离限制。

iSCSI 的缺点是：存储和网络是同一个物理接口，同时协议本身的开销较大，协议本身需要频繁地将 SCSI 命令封装到 IP 包中以及从 IP 包中将 SCSI 命令解析出来，这两个因素都造成了带宽的占用和主处理器的负担。但是，随着专门处理 iSCSI 指令的芯片的开发（解决主处理器的负担问题），以及 10G 以太网的普及（解决带宽问题），iSCSI 将有更好的发展。

(3) IB SAN。无限带宽 (InfiniBand, IB) 是一种交换结构 I/O 技术，其设计思路是通过一套中心机构 (IB 交换机) 在远程存储器、网络以及服务器等设备之间建立一个单一的连接链路，并由 IB 交换机来指挥流量。这种结构设计大大提高了系统的性能、可靠性和有效性，能缓解各硬件设备之间的数据流量拥塞。而在共享总线环境中没有解决好这个问题，因为设备之间的连接都必须通过指定的端口建立单独的链路。

IB 主要支持两种环境：第一，模块对模块的计算机系统（支持 I/O 模块附加插槽）；第二，在数据中心环境中的机箱对机箱的互连系统、外部存储系统和外部局域网 / 广域网访问设备。IB 支持的带宽比现在主流的 I/O 载体（例如，SCSI、FC 等）还要高，另外，由于使用 IPv6 的报头，IB 还支持与传统 Internet/Intranet 设施的有效连接。

IB SAN 采用层次结构，将系统的构成与接入设备的功能定义分开，不同的主机可通过主机通道适配器 (Host Channel Adapter, HCA)、RAID 等网络存储设备利用目标通道适配器 (Target Channel Adapter, TCA) 接入 IB SAN。

IB SAN 的主要特性包括：可伸缩的 Switched Fabric 互连结构；由硬件实现的传输层互连高效、可靠；支持多个虚信道；硬件实现自动的路径变换；高带宽，总带宽随 IB Switch 规模成倍增长；支持 SCSI 远程直接内存存取 (Direct Memory Access, DMA) 协议；具有较高的容错性和抗毁性，支持热插拔。

#### 4. 网络存储技术的选择

网络存储技术的目的是扩大存储能力，提高存储性能。这些存储技术都能提供集中化的数据存储并有效存取文件；都支持多种操作系统，并允许用户通过多个操作系统同时使用数据；都可以从应用服务器上分离存储，并提供数据的高可用性；同时，都能通过集中存储管理来降低长期的运营成本。

### 3.2.5 虚拟存储技术

虚拟存储 (Virtual Storage) 是指把多个存储介质模块 (例如，硬盘、RAID 等) 通过一定的手段集中管理，形成统一管理的存储池 (Storage Pool)，为用户提供大容量、高数据传输性能的存储系统。存储虚拟化是将实际的物理存储实体与存储的逻辑表示实现分离，使用虚拟存储技术，应用服务器只与分配给它们的逻辑卷 (虚卷) 交互，而不用关心其数据是在哪个物理存储实体上。

#### 1. 虚拟存储的分类

按照拓扑结构的不同，虚拟存储可以分为对称式和非对称式两种方式。其中，对称式是指

虚拟存储控制设备与存储软件系统、交换设备集成为一个整体，内嵌在网络数据传输路径中；非对称式是指虚拟存储控制设备独立于数据传输路径之外。

按照实现原理的不同，虚拟存储也可以分为数据块虚拟和虚拟文件系统两种方式。

(1) 数据块虚拟。该存储方式着重解决数据传输过程中的冲突和延时问题，利用虚拟的多端口并行技术，为多个用户提供极高的带宽，最大限度减少延时与冲突的发生。

(2) 虚拟文件系统。该存储方式着重解决大规模网络中文件共享的安全机制问题。通过对不同的站点指定不同的访问权限，保证网络文件的安全。

## 2. 虚拟存储的实现方式

虚拟存储要解决的关键问题是逻辑卷与物理存储实体之间的映射关系，这种映射关系可以在计算机层解决，也可以在存储设备层解决，还可以在存储网络层解决。

(1) 主机级的虚拟化。由安装在应用服务器上的卷管理软件完成存储的虚拟化。基于主机端的虚拟存储几乎都是通过纯软件的方式实现的，这种实现机制不需要引入新设备，也不影响现有存储系统的基本架构，所以实现成本比较低。

(2) 存储设备级的虚拟化。由存储设备的控制器实现存储的虚拟化。这种虚拟存储一般是存储厂商实施的，很可能使用厂商独家的存储产品。这种实现机制主要通过大规模的 RAID 子系统和多个 I/O 通道连接到服务器上，智能控制器提供访问控制、缓存和其他（例如，数据复制等）管理功能。

(3) 网络级的虚拟化。由加入 SAN 的专用装置实现存储虚拟化。这种机制可以管理不同厂商的存储设备，实现 SAN 中所有设备的统一管理，具有较好的开放性。

总之，不管采用何种虚拟存储技术，其目的都是提供一个高性能、安全、稳定、可靠、可扩展的存储网络平台，满足系统的要求。

## 3. 虚拟存储的特点

虚拟存储技术是为了解决复杂、烦琐的存储管理而产生的，广泛应用于数据镜像、数据复制、实时数据恢复、应用集成等方面。虚拟存储主要包含如下几方面的优秀性能：

(1) 虚拟存储提供了一个大容量存储系统集中管理的手段，由网络中的一个环节进行统一管理，避免了由于存储设备扩充所带来的管理方面的麻烦。例如，增加新的存储设备时，只需要管理员对存储系统进行较为简单的配置更改，客户端无需任何操作。

(2) 虚拟存储可以大大提高存储系统整体访问带宽。存储系统是由多个存储模块组成的，而虚拟存储系统可以很好地进行负载平衡，把每一次数据访问所需的带宽合理地分配到各个存储模块上，这样，系统的整体访问带宽就增大了。

(3) 虚拟存储技术为存储资源管理提供了更好的灵活性和兼容性，可以将不同类型的存储设备集中管理使用，保护用户的已有投资。

(4) 虚拟存储技术可以通过管理软件，为网络系统提供一些其他的有用功能，例如，无须服务器的远程镜像和数据快照等。

(5) 虚拟存储技术将计算机的应用系统与存储设备分离，使各种不同的存储设备具有标准的存储特性，而应用系统无须关心数据存储的具体设备，减轻了应用系统的负担。

## 3.3 输入输出系统

输入输出系统简称 I/O 系统, 它由 I/O 设备、I/O 接口 (I/O 控制器)、I/O 控制管理软件等组成。I/O 系统将各种 I/O 设备有效地入计算机系统, 将计算机外部信息输入到计算机内部 (简称为输入操作), 以便进行加工处理; 将计算机内部存储和加工处理的信息输出到计算机之外 (简称为输出操作), 以提供给计算机外部设备使用。

### 3.3.1 输入输出工作方式

在计算机中, I/O 系统可以有 5 种不同的工作方式, 分别是程序控制方式、程序中断方式、DMA 工作方式、通道方式、I/O 处理机。

#### 1. 程序控制方式

程序控制方式是由 CPU 执行一段 I/O 程序来实现主机与外设之间的数据传送, 根据外设的不同性质, 可分为无条件传送和程序查询方式两种。

在无条件传送方式中, I/O 端口总是准备好接收主机的输出数据, 或总是准备好向主机输入数据, 因而 CPU 无须查询外设的工作状态, 而默认外设始终处于准备就绪状态。在 CPU 认为需要时, 随时可直接利用 I/O 指令访问相应的 I/O 端口, 实现与外设之间的数据交换。这种方式的优点是软、硬件结构都很简单, 但要求时序配合精确, 一般的外设难以满足要求。所以, 只能用于对简单开关量的 I/O 控制。

许多外设的工作状态是很难事先预知的, 例如, 用户何时按键, 打印机是否能接收新的打印输出信息等。当 CPU 与外设工作不同步时, 很难确保 CPU 在执行输入操作时, 外设一定是“准备好”的; 而在执行输出操作时, 外设一定是“缓冲器空”的。为了保证数据传送的正确进行, 就要求 CPU 在程序中查询外设的工作状态。如果外设尚未准备就绪, CPU 就循环等待, 只有当外设已做好准备时, CPU 才能执行 I/O 指令进行数据传送, 这就是程序查询方式。由程序主动查询外设, 完成主机与外设间的数据传送, 这种方法简单, 硬件开销小, 但 I/O 能力不高, 严重影响 CPU 的利用率。

#### 2. 程序中断方式

为了提高 I/O 能力和 CPU 的效率, 计算机系统引进了中断方式。程序中断是指计算机执行现行程序的过程中, 出现某些急需处理的异常情况或特殊请求, CPU 暂时中止现行程序 (保护现场), 而转去对随机发生的更紧迫的事件进行处理, 在处理完毕后, CPU 将自动返回原来的程序继续执行 (恢复现场)。整个中断过程大体上可以分为 5 个阶段, 分别是中断请求、中断判优、中断响应、中断处理和中断返回。在系统中具有多个中断源的情况下, 常用的处理方法有多中断信号线法、中断软件查询法、雏菊链法、总线仲裁法和中断向量表法。

当 I/O 系统与外设间用中断控制方式交换数据时, CPU 无须等待也不必去查询 I/O 的状态。进行中断响应时, CPU 保存正在执行程序现场, 转入 I/O 中断服务程序完成与 I/O 系统的数据交换, 然后返回原主程序继续执行。与程序控制方式相比, 中断方式因为 CPU 无须等待而提高了效率。

### 3. DMA 工作方式

DMA 工作方式是为了在主存与外设之间实现高速、批量数据交换而设置的，使用 DMA 控制器（Direct Memory Access Controller, DMAC）来控制和管理数据传输。DMAC 和 CPU 共享系统总线，并且具有独立访问存储器的能力。在进行 DMA 时，CPU 放弃对系统总线的控制而由 DMAC 控制总线；由 DMAC 提供存储器地址及必需的读写控制信号，实现外设与存储器之间的数据交换。

DMAC 获取总线的方式主要有三种，分别是 CPU 停止访问主存法（暂停方式）、存储器分时法（共享方式）和周期挪用法（周期窃取方式）。DMA 工作方式具有下列特点：

- (1) 它使主存与 CPU 的固定联系脱钩，主存既可被 CPU 访问，又可被外设访问。
- (2) 在数据块传送时，主存地址的确定、传送数据的计数等都由硬件电路直接实现。
- (3) 主存中要开辟专用缓冲区，及时供给和接收外设的数据。
- (4) DMA 传送速度快，CPU 和外设并行工作，提高了系统的效率。
- (5) DMA 在传送前要通过程序进行预处理，结束后要通过中断方式进行后处理。

### 4. 通道方式

通道是一种高级的 I/O 控制部件，它在一定的硬件基础上利用软件手段实现对 I/O 的控制和传送，更多地免去了 CPU 的介入，从而使主机和外设的并行工作程度更高。当然，通道并不能完全脱离 CPU，它还要受到 CPU 的管理，比如启动、停止等，而且通道还应该向 CPU 报告自己的状态，以便 CPU 决定下一步的处理。

按照所采取的传送方式，可将通道分为字节多路通道、选择通道和数组多路通道。

(1) 字节多路通道。这是一种简单的共享通道，用于连接与管理多台低速设备，以字节交叉方式传送信息。

(2) 选择通道，又称为高速通道。该通道在物理上也可以连接多个设备，但这些设备不能同时工作，在一段时间内通道只能选择一台设备进行数据传送，此时该设备可以独占整个通道。

(3) 数组多路通道。该通道是把字节多路通道和选择通道的特点结合起来的一种通道结构，它有多个子通道，既可以执行多路通道程序，即像字节多路通道那样，所有子通道分时共享总通道，又可以用选择通道那样的方式成组地传送数据；既具有多路并行操作的能力，又具有很高的数据传输速率，使通道的效率充分得到发挥。

### 5. I/O 处理机

I/O 处理机也称为外围处理机，具有丰富的指令系统和完善的中断系统。专用于大型、高效的计算机系统处理外围设备的 I/O，并利用共享存储器或其他共享手段与主机交换信息，从而使大型计算机系统更加高效地工作。

I/O 处理机具有比较丰富的指令系统，结构接近于一般的处理机，可以有自己的局部存储器。I/O 处理机除了能够完成通道的全部功能外，还可以进行码制转换、数据校验和校正、故障处理、文件管理、诊断和显示系统状态、处理人机对话。网络或远程终端可以直接连接到 I/O 处理机上，由 I/O 处理机完成远程用户服务工作。I/O 处理机还可以根据需要完成分配给它的其他任务，例如，进行数据库和知识库的管理工作等。

I/O 处理机基本上是独立于中央处理机异步工作的, 它可以与中央处理机共享主存, 也可以有自己独立的存储器, 不共享主存。每台 I/O 处理机可以有自己独立的运算部件和指令控制部件, 也可以由多个 I/O 处理机共享同一个运算部件和指令控制部件。

### 3.3.2 总线

总线是一组能为多个部件分时共享的公共信息传送线路。共享是指总线上可以挂接多个部件, 各个部件之间相互交换的信息都可以通过这组公共线路传送; 分时是指同一时刻只允许有一个部件向总线发送信息, 若出现两个或两个以上部件同时向总线发送信息, 则会导致信号冲突。但在同一时刻, 允许多个部件同时从总线上接收相同的信息。

#### 1. 总线的分类

按总线相对于 CPU 或其他芯片的位置, 可将总线分为内部总线和外部总线两种。在 CPU 内部, 寄存器之间和算术逻辑部件 (ALU) 与控制部件之间传输数据所用的总线称为内部总线; 外部总线是指 CPU 与内存 RAM、ROM 和输入/输出设备接口之间进行通信的通路。由于 CPU 通过总线实现程序取指令、内存/外设的数据交换, 在 CPU 与外设一定的情况下, 总线速度是制约计算机整体性能的最大因素。

总线有不同的分类方式, 主要有如下 4 种分类方法:

(1) 按总线功能, 可分为地址总线、数据总线、控制总线三类。地址总线用来传送地址信息, 数据总线用来传送数据信息, 控制总线用来传送各种控制信号。例如, 工业标准结构 (Industrial Standard Architecture, ISA) 总线共有 98 条线, 其中数据线有 16 条、地址线有 24 条, 其余为控制信号线、接地线和电源线。

(2) 按总线在微机系统中的位置, 可分为机内总线和机外总线两种。上面所说的总线都是机内总线, 而机外总线是指与外部设备接口相连的, 是一种外设的接口标准。

(3) 按总线功用, 主要分为局部总线、系统总线、通信总线三类。局部总线是在传统的 ISA 总线和 CPU 总线之间增加的一级总线或管理层; 系统总线是计算机系统内部各部件 (插板) 之间进行连接和传输信息的一组信号线, 例如, 微通道结构 (Micro Channel Architecture, MCA)、视频电子标准协会 (Video Electronic Standard Association, VESA)、加速图形接口 (Accelerate Graphical Port, AGP) 等; 通信总线是计算机系统之间或计算机系统与其他系统 (例如, 远程通信设备、测试设备等) 之间进行通信的一组信号线。

(4) 按总线中数据线的多少, 可分为并行总线和串行总线两类。并行总线是含有多条双向数据线的总线, 它可以实现一个数据的多位同时传输, 总线中数据线的数量决定了可传输一个数据的最大位数 (一般为 8 的倍数), 其优点是数据传输速率高; 串行总线是只含有一条双向数据线或两条单向数据线的总线, 可以实现一个数据的各位按照一定的速度和顺序依次传输, 适用于远距离通信。

#### 2. 总线标准

总线标准是指计算机部件各生产厂家都需要遵守的总线要求, 从而使不同厂家生产的部件能够互换。总线标准主要规定总线的机械结构规范、功能结构规范和电气规范。总线标准可以

分为正式标准和工业标准，其中正式标准是由 IEEE 等国际组织正式确定和承认的标准；工业标准也称为事实标准，是首先由某一厂家提出，然后得到其他厂家广泛使用的标准。

### 3. 总线的性能指标

通常，总线规范中会详细描述总线各方面的特性，包括物理特性、功能特性、电气特性和时间特性。物理特性又称为机械特性，它规定了总线的线数，以及总线的插头、插座的形状、尺寸和信号线的排列方式等要素；功能特性描述总线中每一根线的功能；电气特性定义了每根线上信号的传递方向及有效电平范围；时间特性规定了每根线在什么时间有效以及不同信号之间相互配合的时间关系。

总线的主要性能指标包括如下 5 种：

(1) 总线宽度。总线宽度指的是总线的线数，它决定了总线所占的物理空间和成本。对总线宽度最直接的影响是地址线和数据线的数量。主存空间和 I/O 空间的扩充使地址线数量增加，并行传输要求有足够的数据线。例如，32 位的 PCI 总线允许寻址的主存空间的大小为  $2^{32}=4\text{G}$  个单元。

(2) 总线带宽。总线带宽指总线的最大数据传输速率，即每秒传输的字节数，单位可以用字节/秒 (B/s) 表示。在同步通信中，总线的带宽与总线时钟密不可分，总线时钟频率的高低决定了总线带宽的大小：

$$\text{总线带宽} = \text{总线宽度} \times \text{总线频率}$$

总线的实际带宽还会受到总线长度（总线延迟）、总线负载、总线收发器性能等多方面因素的影响。例如，假设某系统总线在一个总线周期中并行传输 4B 信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10MHz。此时，时钟周期  $T=1/10\text{M}=0.1\mu\text{s}$ ，总线周期  $=2T=0.2\mu\text{s}$ ，则总线带宽为  $4/0.2=20\text{MB/s}$ 。

(3) 总线负载。总线负载是指连接在总线上的最大设备数量。大多数总线的负载能力是有限的。

(4) 总线分时复用。总线分时复用是指在不同时段利用总线上同一个信号线传送不同信号。例如，地址总线 and 数据总线共用一组信号线。采用这种方式的目的是减少总线数量，提高总线的利用率。

(5) 总线猝发传输。猝发式数据传输是一种总线传输方式，即在一个总线周期中可以传输存储地址连续的多个数据。

需要注意的是，除了上述性能指标外，总线是否具有即插即用功能，是否支持总线设备的热插拔，是否支持多主控设备，是否具有错误检测能力，是否依赖于特定 CPU 等，也是评价总线性能的指标。

### 3.3.3 I/O接口

主机和外设具有各自的工作特点，无论在信息形式还是在工作速度方面都具有很大的差异，I/O 接口（简称为接口）是为了解决这些差异而设置的。I/O 接口也称为 I/O 控制器，它是主机和外设（外部设备）之间的界面，通过接口可以实现主机和外设之间的信息交换。

### 1. I/O 接口的功能

通常, I/O 接口的功能主要包括如下 5 个方面:

(1) 实现主机和外设的通信联络控制。接口中的同步控制电路用来解决主机与外设的时间配合问题。通过联络信息可以决定不同工作速度的外设和主机之间交换信息的最佳时刻, 以保证整个计算机系统能统一协调地工作。

(2) 进行地址译码和设备选择。当 CPU 送来选择外设的地址码后, 接口必须对地址进行译码以产生设备选择信息, 使主机能和指定外设交换信息。

(3) 实现数据缓冲。数据缓冲寄存器用于数据的暂存, 以避免丢失数据。在传送过程中, 先将数据送入数据缓冲寄存器中, 然后再送到输出设备或主机中。

(4) 数据格式的变换。为了满足主机或外设的各自要求, 接口电路中必须具有各类数据相互转换的功能。例如, 并串转换、数模转换等。

(5) 传递控制命令和状态信息。当 CPU 要启动某一外设时, 通过接口中的命令寄存器向外设发出启动命令; 当外设准备就绪时, 则将“准备好”的状态信息送回接口中的状态寄存器, 为 CPU 提供外设已经具备与主机交换数据的条件的反馈信息; 当外设向 CPU 提出中断请求和 DMA 请求时, CPU 也应有相应的响应信号反馈给外设。

### 2. 接口的分类

I/O 接口有多种分类方法: 按 I/O 的信号分类, 可分为数字接口和模拟接口; 按通用性分类, 可分为通用接口和专用接口; 按功能选择的灵活性分类, 可分为可编程接口和不可编程接口; 根据外部设备与 I/O 模块交换数据的方式, 可分为串行接口和并行接口, 串行接口一次只能传送 1 位信息, 而并行接口一次可传送多位信息。

串行通信又可分为异步通信方式和同步通信方式两种。

(1) 异步通信。该方式在发送字符时, 所发送字符之间的时间间隔可以是任意的。接收端必须时刻做好接收的准备, 发送端可以在任意时刻开始发送字符, 因此必须在每一个字符的开始和结束的地方加上标志, 即加上开始位和停止位, 以便使接收端能够正确地将每一个字符接收下来。异步通信的好处是通信设备简单、便宜, 但传输效率较低。

(2) 同步通信。该方式要求收发双方具有同频同相的同步时钟信号, 只需在传送报文的最前面附加特定的同步字符, 使收发双方建立同步, 此后, 便在同步时钟的控制下逐位发送和接收。

### 3. 常见接口

常见的设备接口有以下几种:

(1) IDE。IDE 是最常用的磁盘接口, 分为普通 IDE 接口和增强型 IDE (EIDE) 接口。普通 IDE 接口的数据传输速率不超过 1.5Mb/s, 数据传输宽度为 8 位, 最多可连接 4 个 IDE 设备, 每个 IDE 硬盘容量不超过 528MB; EIDE 接口的数据传输速率可达 150Mb/s, 数据传输宽度为 32 位。

(2) ATA (Advanced Technology Attachment)。ATA 发展至今经过了多次修改和升级, 新一代的接口都建立在前一代标准之上, 并保持着向后兼容性。ATA-7 是 ATA 接口的最后一个版本,

也称为 Ultra DMA 133, 支持 1064Mb/s (133MB/s) 的数据传输速率。

(3) SATA (Serial ATA)。SATA 是一种基于行业标准的串行硬件驱动器接口。与 ATA 相比, SATA 规范将硬盘的外部传输速率理论值提高到了 1200Mb/s (150MB/s), 而随着后续版本的发展, SATA 接口的速率已经扩展到 3000Mb/s, 有望达到 4800Mb/s。SATA 接口需要硬件芯片的支持, SATA 的优势是支持热插拔、传输速度快、执行效率高。

(4) eSATA (external SATA)。eSATA 支持 3.2Gb/s 的传输速率, 是一种扩展的 SATA 接口, 用来连接外部 SATA 设备。eSATA 是基于标准的 SATA 线缆和接口, 连接处加装了金属弹片来保证物理连接的稳固性, eSATA 线缆能够插拔数千次。

(5) SCSI。SCSI 是大容量存储设备、音频设备和 CD-ROM 驱动器的一种标准。SCSI 接口通常被看作一种总线, 可用于连接多个外设, 这些 SCSI 设备以菊花链形式接入, 并被分配了唯一的 ID 号, 其中最后一个 ID 分配给 SCSI 控制器。SCSI 设备彼此独立运作, 相互之间可以交换数据, 也可以和主机进行交互。数据以分组消息的形式进行传输。目前, SCSI 的最大同步传输速率为 5Gb/s (640MB/s)。

(6) PCMCIA。PCMCIA 是一种广泛用于笔记本电脑的接口标准, 体积小, 扩展较方便、灵活。最初 PCMCIA 主要用于笔记本电脑扩展内存, 目前常用作一种存储器卡接口或进行传真、调制解调器功能扩展接口。现在, 用 PCMCIA 代表个人计算机存储卡国际协会, 而 PCMCIA 接口更名为 PC Card 接口。

(7) IEEE-1394。1394 也被称为 i.Link 或 FireWire, 是由 IEEE 于 1995 年发布的, 它的最初版本被称为 1394a, 初始数据传输速率为 200Mb/s (25MB/s), 现在 (Fire Wire 800) 的数据传输速率为 800Mb/s, 而新的 1394b (Fire Wire 3200) 有望支持 3200Mb/s 的数据传输速率。1394 是构建在菊花链或树状拓扑结构上的, 它支持 63 个节点, 每个节点可以支持多达 16 台设备的菊花链, 并且还支持最多 1023 条桥接的总线。1394 能够在同一条总线上支持不同速率的设备, 支持设备的热插拔。

(8) USB。USB 接口是一种串行总线式的接口, 在串行接口中可达到较高的数据传输速率, 并且也允许设备以菊花链形式接入, 最多可连接 127 个设备。USB 的最大特点是允许热插拔, 目前在便携式计算机和台式计算机中已成为标准配置。许多数码相机、闪存、视频摄像头以及打印机等都可通过 USB 接口接入计算机。USB 1.0 的速度是 12Mb/s (1.5MB/s), USB 2.0 的速度达到了 480Mb/s, USB 3.0 的速度可达 4.8Gb/s。

#### 4. 端口

I/O 端口 (简称为端口) 是指接口电路中可以 CPU 直接访问的寄存器, 由若干个端口加上相应的控制逻辑电路组成接口。接口包含数据端口、命令端口和状态端口。CPU 通过输入指令可以从端口读取信息, 通过输出指令可以把信息写入端口。CPU 对不同端口的操作有所不同, 有的端口只能写或只能读, 有的端口既可读又可写。

端口编址的方式分为独立编址和统一编址两种。

(1) 独立编址又称为 I/O 映射。在该编址方式中, 主存地址空间和端口地址空间相对独立, 分别单独编址。在指令系统中设置专门的 I/O 指令, 当 CPU 使用 I/O 指令时, 其指令的地址字

段直接或间接地指示出端口地址。

(2) 统一编址又称为存储器映射。在该编址方式中, 端口地址和主存单元地址是统一编址的, 把 I/O 接口中的端口视为同主存单元一样进行访问, 不设置专门的 I/O 指令, 就用一般的数据传送类指令来实现 I/O 操作。

## 3.4 指令系统

指令是指示计算机执行某些操作的命令, 一台计算机的所有指令的集合构成指令系统, 也称为指令集。指令系统是计算机系统中软件与硬件分界面的一个主要标志。无论结构多么复杂、功能多么强大的软件, 凡是能够在机器上直接运行的目标程序, 都是由若干条机器指令组成的。

在计算机系统中, 指令系统的设计是非常关键的环节, 通常由软件设计人员和硬件设计人员共同完成。

### 3.4.1 基本指令系统

如果把计算机系统所要实现的任务分解成若干个基本功能, 那么在这些基本功能中, 只有极少数的基本功能必须用硬件的指令系统来实现, 而大多数基本功能既可以用硬件的指令系统来实现, 也可以用软件的一段子程序(微程序)来实现。

在实际的系统设计时, 某项功能是用硬件实现还是用软件实现, 主要考虑三个方面的因素, 分别是速度、价格和灵活性。用硬件的指令来实现, 速度快、价格高、灵活性差; 用软件的子程序来实现, 速度慢、价格低、灵活性好。

#### 1. 设计要求

设计指令系统时的基本要求包括指令系统的完整性、规整性、高效率 and 兼容性。

(1) 完整性。完整性是指作为通用计算机所应该具备的基本指令种类。

(2) 规整性。规整性主要包括对称性和均匀性。其中, 对称性是指各种与指令系统有关的数据存储设备的使用、操作码的设置等都要对称(例如, 所有通用寄存器要同等对待); 均匀性是指对于各种不同的数据类型、字长、操作种类和数据存储设备, 指令的设置要同等对待(例如, 某机器有 5 种数据表示、4 种字长、8 种数据存储设备的有效排列, 则设计加法指令时, 指令的种类应该有  $5 \times 4 \times 8 = 160$  种两地址加法指令)。

(3) 高效率。高效率是指指令的执行速度要快、使用频率要高。对于那些使用频率比较低的指令, 要尽量少设置; 对于那些比较复杂但又必须设置的指令, 可以采用微程序来实现, 以减少硬件的复杂程度。

(4) 兼容性。兼容性是计算机系统的生命力所在。如果没有兼容性, 大量的系统软件和各种应用软件就无法集成, 计算机也就没有了市场。

#### 2. 基本指令

通用计算机系统的基本指令有数据传送类指令、运算类指令、程序控制类指令、I/O 指令、处理机控制和调试指令。

(1) 数据传送类指令。数据传送类指令是最基本的指令类型，主要用于实现寄存器与寄存器之间、寄存器与主存单元之间，以及两个主存单元之间的数据传送。数据传送类指令的种类由三个主要因素决定，分别是数据存储设备的种类、数据传送单位和采用的寻址方式。数据传送类指令又可以细分为一般传送指令、堆栈操作指令和数据交换指令。

(2) 运算类指令。运算类指令又分为算术运算指令、逻辑运算指令和移位指令。其中，移位指令又可分为算术移位、逻辑移位和循环移位。运算类指令在整个指令系统中应该占有比较大的比重（例如，超过 30%）。如果所占比重过小，就会影响整个计算机系统的性能。设计运算类指令，主要考虑操作种类、数据表示、数据长度、数据存储设备，以及它们的组合。

(3) 程序控制类指令。程序控制类指令用于控制程序的执行顺序，并使程序具有测试、分析与判断的能力，主要包括三类，即转移指令（包括无条件转移和有条件转移）、程序调用和返回指令、循环控制指令。其中，前两类指令在一般计算机中是必须具备的，最后一类指令用于对循环程序进行优化。

(4) I/O 指令。I/O 指令用来实现主机与外部设备之间的信息交换，包括 I/O 数据、主机向外设发控制命令或外设向主机报告工作状态等。I/O 指令通常比较简单，采用单的直接寻址方式，数据字长一般以字节为单位。在多用户或多任务环境下，I/O 指令属于特权指令，必须在管态下运行。当程序需要进行 I/O 操作时，用系统调用进入操作系统，由操作系统对设备统一进行管理。

(5) 处理机控制和调试指令。在一般的计算机系统中，处理机有两个状态，即管态和用户态，或称为主态和从态。在通用计算机系统中，可以将指令分为两大类，即一般指令和特权指令。在用户态下，只能使用一般指令；在管态下，可以使用特权指令。特权指令主要用于处理机状态的设置和管理、系统硬件和软件资源的管理、进程的管理等。

### 3.4.2 复杂指令系统

为了增强指令的功能，设置一些功能复杂的指令，把一些原来由软件实现的、常用的功能改用硬件来实现，这种计算机系统称为复杂指令系统计算机（Complex Instruction Set Computer, CISC），简称复杂指令系统。

#### 1. CISC 的特点

CISC 的主要特点如下：

- (1) 指令数量众多。指令系统拥有大量的指令，通常有 100 ~ 250 条左右。
- (2) 指令使用频率相差悬殊。最常使用的是一些比较简单的指令，仅占指令总数的 20%，但在程序中出现的频率却占 80%，而大部分复杂指令却很少使用。
- (3) 支持很多种寻址方式。支持的寻址方式通常有 5 ~ 20 种。
- (4) 变长的指令。指令长度不是固定的，变长的指令增加了指令译码电路的复杂性。
- (5) 指令可以对主存单元中的数据直接进行处理。典型的 CISC 通常都有指令能够直接对主存单元中的数据进行处理，其执行速度较慢。
- (6) 以微程序控制为主。CISC 的指令系统很复杂，难以用硬布线逻辑（组合逻辑）电路实现控制器，通常采用微程序控制。

## 2. 目标程序的优化

目标程序是由指令直接组成的,可以在处理机中直接执行,而面向目标程序优化指令系统是提高计算机系统性能的最直接的方法。优化目标程序主要有两个目的:一是缩短程序的长度,即减少程序的空间开销;另一个是缩短程序的执行时间,即减少程序的时间开销。

优化目标程序的主要途径是增强指令的功能,包括数据传送类指令、运算类指令和程序控制类指令。具体方法为:对大量的程序及其执行情况进行统计分析,找出使用频率高、执行时间长的指令和指令串;对于使用频率高的指令,用硬件加快其执行,以缩短整个程序的执行时间;对于使用频率高的指令串,用一条新的指令来代替它,这样既能缩短整个程序的执行时间,也能缩短整个程序的长度,从而减少程序的空间开销。

## 3. 对高级语言和编译程序的支持

用高级语言编写的程序经编译程序编译后生成的目标程序,与直接用机器语言或汇编语言编写的程序相比,时空开销都要大得多。改进指令系统,增加对高级语言和编译程序的支持,缩小高级语言与机器语言之间的差距,从而提高整个计算机系统的性能。

面向高级语言和编译程序增强指令系统的途径主要有两个:一是增强对高级语言和编译程序支持的指令的功能,增强体系结构的规整性,减少体系结构中各种例外情况;二是设计高级语言计算机,在这种计算机中,用高级语言编写的程序不需要经过编译,直接由机器的硬件来执行,例如,LISP 计算机和 PROLOG 计算机等。

## 4. CISC 的缺陷

CISC 主要存在如下三个方面的问题:

(1) 80-20 规律。在 CISC 中,指令使用频率不均衡,即各种指令的使用频率相差悬殊。大量统计数字表明,大约有 20% 的指令使用频率比较高,占据了 80% 的处理机时间。换句话说,有 80% 的指令只在 20% 的处理机运行时间内才被用到。

(2) CISC 结构指令系统的复杂性给 VLSI (超大规模) 设计增加了负担,不利于单片集成。但随着 VLSI 集成度迅速提高,生产单芯片处理机已成为现实。在单芯片处理机内,希望采用规整的硬布线控制逻辑,而不是用微程序。

(3) 软硬件的功能分配问题。为了支持目标程序的优化,支持高级语言和编译程序,在 CISC 中增加了许多复杂的指令。这些复杂指令简化了目标程序,缩小了高级语言与机器指令之间的语义差距,但这是以增加硬件的复杂程度、指令执行周期长为代价的。

### 3.4.3 精简指令系统

精简指令系统计算机 (Reduced Instruction Set Computer, RISC) 简称精简指令系统,该系统不是简单地把指令系统进行简化,而是通过简化指令的途径使计算机的结构更加简单、合理,以减少指令的执行周期数,从而提高运算速度。

#### 1. RISC 的特点

RISC 要求指令系统简化,操作在单周期内完成,指令格式力求一致,寻址方式尽可能减少,

并提高编译的效率,最终达到加快机器处理速度的目的。RISC 的主要特点如下:

(1) 指令数量少。优先选取使用频率最高的一些简单指令和常用指令,避免使用复杂指令。只提供了 LOAD(从存储器中读数)和 STORE(把数据写入存储器)两条指令对存储器操作,其余所有的操作都在 CPU 的寄存器之间进行。

(2) 指令的寻址方式少。通常只支持寄存器寻址、立即数寻址和相对寻址方式。

(3) 指令长度固定,指令格式种类少。RISC 指令数量少、格式种类少、相对简单,其指令长度固定,指令之间各字段的划分比较一致,译码相对容易。

(4) 以硬布线逻辑控制为主。为了提高操作的执行速度,通常采用硬布线逻辑(组合逻辑)来构建控制器。

(5) 单周期指令执行,采用流水线技术。因为简化了指令系统,很容易利用流水线技术,使得大部分指令都能在一个机器周期内完成。少数指令可能需要多周期,例如,LOAD/STORE 指令因为需要访问存储器,其执行时间就会长一些。

(6) 优化的编译器。RISC 的精简指令集使编译工作简单化。因为指令长度固定、格式种类少、寻址方式少,编译时不必在具有相似功能的许多指令中进行选择,也不必为寻址方式的选择而费心,同时易于实现优化,从而可以生成高效率执行的机器代码。

(7) CPU 中的通用寄存器数量多,一般在 32 个以上,有的可达上千个。

大多数 RISC 采用了 Cache 方案,使用 Cache 来提高取指令的速度。而且,有的 RISC 使用两个独立的 Cache 来改善性能,一个称为指令 Cache,另一个称为数据 Cache。这样,取指令和取数据可以同时进行,互不干扰。

## 2. RISC 的核心思想

计算机执行一个程序所用的时间可表示为:

$$P = I \times \text{CPI} \times T$$

其中, $P$ 是执行这个程序所使用的总时间, $I$ 是这个程序所需执行的总的指令条数,CPI 是每条指令执行的平均周期数, $T$ 是一个周期的时间长度。

(1) 由于 RISC 的指令都比较简单,CISC 中的一条复杂指令所完成的功能在 RISC 中可能要用几条指令才能实现。对于同一个源程序,编译后生成的动态目标代码,显然 RISC 的要比 CISC 的多。但是,由于 CISC 中复杂指令使用的频率很低,程序中使用的绝大多数指令都是与 RISC 一样的简单指令,因此,实际上的统计结果表明,RISC 的  $I$  长度只比 CISC 的长 20%~40%。

(2) 由于 CISC 一般是用微程序实现的,一条指令往往要用好几个周期才能完成,一些复杂指令所需要的周期数就更多。根据统计,大多数 CISC 的指令平均执行周期数(CPI)为 4~10;而 RISC 的大多数指令都是单周期执行的,它们的 CPI 应该是 1,但是,由于 RISC 中还有 LOAD 和 STORE 指令,以及少数复杂指令,所以,其 CPI 要略大于 1。

(3) 由于 RISC 一般采用硬布线逻辑实现,指令要实现的功能都比较简单,所以,CISC 的  $T$  通常是 RISC 的 3 倍左右。

综合以上三点,可以大致计算出,RISC 的处理速度要比同规模的 CISC 提高 3~5 倍。其中的关键在于 RISC 的指令平均执行周期数(CPI)减小了,这正是 RISC 设计思想的精华。减

小 CPI 是多个方面共同努力的结果。在硬件方面,采用硬布线逻辑控制,减少指令和寻址方式的种类,使用固定的指令格式,采用 LOAD/STORE 结构,指令执行过程中设置多级流水线等;软件方面则十分强调优化编译技术的作用。

### 3. RISC 的关键技术

RISC 要达到很高的性能,必须有相应的技术支持。目前,在 RISC 处理机中采用的主要技术有如下几种:

(1) 延迟转移技术。在 RISC 处理机中,指令一般采用流水线方式工作。流水线技术所面临的一个问题就是转移指令的出现,这样有可能使流水线断流。其解决办法是采用延迟转移技术,该方法是在转移指令之后插入一条有效的指令,使得转移指令好像被延迟执行了。

(2) 指令取消技术。采用指令延迟技术,当遇到条件转移指令时,调整指令序列非常困难,在许多情况下找不到可以用来调整的指令。有些 RISC 处理机采用指令取消技术。在使用指令取消技术的处理机中,所有转移指令和数据变换指令都可以决定下面待执行的指令是否应该取消。如果指令被取消,其效果相当于执行了一条空操作指令,不影响程序的运行环境。为了提高程序的执行效率,应该尽量少取消指令,以保持指令流水线处于充满状态。

(3) 重叠寄存器窗口技术。在处理机中设置一个数量比较大的寄存器堆,并把它划分成很多个窗口。每个过程使用其中相邻的三个窗口和一个公共的窗口,而在这些窗口中,有一个窗口是与前一个过程共用的,还有一个窗口是与下一个过程共用的。与前一个过程共用的窗口可以用来存放前一个过程传送给本过程的参数,同时也存放本过程传送给前一个过程的计算结果;与下一个过程共用的窗口可以用来存放本过程传送给下一个过程的参数,同时也存放下一个过程传送给本过程的计算结果。

(4) 指令流调整技术。为了使 RISC 处理机中的指令流水线高效率地工作,尽量不断流,编译器必须分析程序的数据流和控制流,当发现指令流有断流可能时,要调整指令序列。对有些可以通过变量重新命名来消除的数据相关,要尽量消除。这样,可以提高流水线的执行效率,缩短程序的执行时间。

(5) 逻辑实现以硬件为主,固件为辅。RISC 主要采用硬布线逻辑来实现指令系统。对于那些必需的复杂指令,也可用固件实现。

在实际应用中,商品化的 RISC 机器并不是纯粹的 RISC。为了满足应用的需要,实际应用的 RISC 除了保持 RISC 的基本特色之外,还必须辅以一些必不可少的复杂指令(例如,浮点运算、十进制运算指令等)。这种机器实际上是在 RISC 基础上实现了 RISC 与 CISC 的完美结合。

## 3.5 多处理机系统

流水线计算机通过多级流水的同时操作来获得高性能,但只能执行单个程序。而且,由于器件本身的限制,任何处理机的速度都是有限的。要想突破这个限制,就必须使用多个处理机并行执行。多处理机结构由若干个独立的处理机组成,每个处理机能够独立执行自己的程序。多处理机结构属于 MIMD 结构,处理机之间按某种形式互连,从而实现程序之间的数据交换和同步。

### 3.5.1 多处理机系统概述

多处理机（Multi Processor）具有两个或两个以上的处理机，共享 I/O 子系统，在操作系统统一控制下，通过共享主存或高速通信网络进行通信，协同求解一个个复杂的问题。多处理机通过利用多台处理机进行多任务处理来提高速度，利用系统的重组能力来提高可靠性、适应性和可用性。

#### 1. 多处理机与并行处理机的比较

并行处理机是基于 SIMD 结构的，即在增加硬件资源的同时对多个数据进行类似的处理。并行处理机只有一个控制器，但有多个处理单元。在控制器的控制下，所有处理单元进行同样的运算，不同的是进行运算的数据不同。如果是非并行计算的程序部分，则由控制器完成。因此，并行处理机特别适合于矩阵（向量）计算，它也被称为矩阵处理机（向量处理机或阵列处理机）。

多处理机与并行处理机相比，有很大的差别，其根源就在于两者的并行性的层次不同。下面，从 5 个方面对并行处理机和多处理机进行比较分析。

（1）结构灵活性。并行处理机的结构主要是针对数组（向量）处理算法而设计的，其特点是处理单元众多，但只需设置有限和固定的处理机之间的互连通路，即可满足高并行性算法的需要；多处理机则应有较强的通用性，能适应更为多样的算法，具备更为灵活多变的系统结构，以实现各种复杂的处理机之间的互连模式，同时，还要解决共享资源的冲突问题。

（2）程序并行性。并行处理机实现操作一级的并行，程序并行性的识别较易实现，可由程序员在编制程序中加以掌握，或由向量化编译程序协助；多处理机实现任务级的并行，再加上系统通用性的要求，就使程序并行性的识别难度较大。

（3）并行任务派生。并行处理机采用 SIMD 方式，由指令本身就可以启动多个处理部件并行工作；多处理机采用 MIMD 方式，一个程序中存在多个并发的程序段，需要专门的指令来表示它们的并发关系以控制并发执行，以便一个任务开始被执行时，就能派生出可与它并行执行的另一些任务。这个过程称为并行任务派生。

（4）进程同步。并行处理机所有处于活动状态的处理单元同时执行共同的指令操作，受同一个控制器控制，工作自然是同步的；多处理机在同一时刻，不同的处理机执行着不同的指令，由于执行时间互不相等，故它们的工作进度不会也不必保持相同。

（5）资源分配和进程调度。并行处理机的处理单元数目是固定的，且受同一控制器的控制，程序员只能利用屏蔽手段来设置部分处理单元为不活动状态，以改变实际参加操作的处理单元数目；多处理机需要使用的处理机数目没有固定要求，各个处理机进入或退出任务的时刻互不相同，所需共享资源的品种、数量又随时变化，因此，存在资源分配和进程调度问题，该问题能否很好地解决，直接影响整个多处理机系统的工作效率。

#### 2. 多处理机系统的分类

多处理机有多个处理单元，这就产生了多个处理单元如何访问内存的问题。通常有两种方式：共享存储方式和分布式存储方式。

### 1) 共享存储方式

共享存储方式的多处理机有公共的共享存储器 (Shared Memory, SM), 各处理机之间通过互连网络共享 SM, 并使用 SM 传递共享公共信息和参数等, 如图 3-6 所示。

采用共享存储访问方式的多处理机系统称为紧耦合系统或直接耦合系统。紧耦合系统的每个处理机可自带局部存储器, 也可自带 Cache, 存储器模块可采用流水工作方式。例如, SMP 就属于紧耦合系统。

按所用处理机类型是否相同及对称, 紧耦合系统又可分为同构或异构, 以及对称或非对称的形式。常见的组合是同构对称式和异构非对称式多处理机系统。紧耦合系统的特点是容易管理和利用资源, 程序员没有划分数据的负担, 编程比较容易, 能加快大程序的运行速度, 常适用于多用户的一般应用和分时应用。但是紧耦合系统的处理机数目有限, 扩充比较困难。

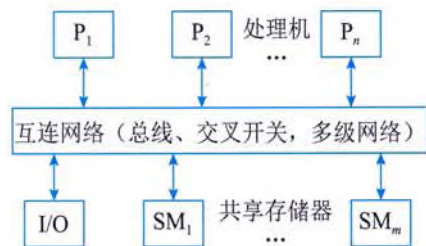


图 3-6 共享存储多处理机模型

### 2) 分布式存储方式

在分布式存储方式的多处理机中, 每个处理机独占本地存储器 (Local Memory, LM), 各处理机通过互连网络相连, 更像计算机网络的结构, 如图 3-7 所示。

采用分布式存储访问方式的多处理机系统称为松耦合系统或间接耦合系统。松耦合系统的每个处理机带有一个 LM 和一组 I/O 设备。例如, MPP 就属于松耦合系统。松耦合系统结构灵活、容易扩充, 但难以在各个处理机之间实现复杂数据结构的数据传送, 任务动态分配复杂, 现有软件可继承性差, 需要设计新的并行算法。松耦合系统较适合粗粒度的并行计算。

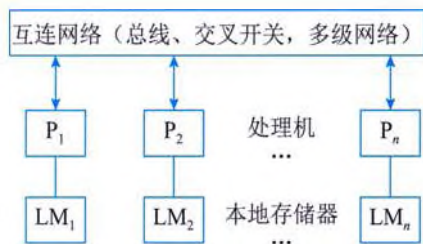


图 3-7 分布式存储多处理机模型

## 3.5.2 海量并行处理结构

MPP 系统的定义随着时间推移在不断地变化。按照当前的标准, 具有几百或几千台处理机的任何机器就是 MPP。显然, 随着计算机技术的快速发展, 对并行度的要求会愈来愈高。MPP 系统最重要的特点是进行大规模并行处理。

在 MPP 系统中, 用的是超大规模集成电路 (Very Large Scale Integrated Circuits, VLSIC)、硅片、砷化镓 (一种半导体材料, 性能比硅更优良) 技术、高密度组装和光技术, 采用可扩展技术、共享虚拟存储技术、容许时延技术、多线程技术的系统结构。

MPP 采用分布式存储方式, 这种方式可以使系统容易扩展, 但因为各处理机不能直接访问非本地存储器, 只能使用消息机制来进行共享, 这就使得编程困难, 并且增加了通信开销。为了解决这个问题, 专家们引入了虚拟共享存储器 (Shared Virtual Memory, SVM) 或共享分布存储器 (Distributed Shared Memory, DSM) 技术。

SVM 是在基于分布存储器的多处理机上, 实现物理上分布但逻辑上共享的存储系统。其基本思想是, 将物理上分散的各个处理机所拥有的 LM, 在逻辑上加以统一编址, 形成一个统一的虚拟地址空间来实现存储器的共享。每个处理机可以访问全局存储器的任一位置, 用户可以

把它当成全局 SM。这样，用户以前在紧耦合系统上编写的程序就可以不加修改地在 SVM 系统上运行，这给软件的移植带来了方便，同时解决了难以对复杂数据结构进行传递和难以进行进程迁移（Process Migration）的问题。

显然，SVM 系统可以兼具紧耦合系统和松耦合系统的优点。目前，实现 SVM 系统的途径主要有如下三种，但现有的 SVM 系统大多采用前两种途径。

（1）硬件实现。将传统的 Cache 技术扩展应用到松耦合系统中，这种途径需要在现有的松耦合系统上增加专用部件，以取得高效的实现。

（2）操作系统和库实现。通过虚拟存储管理机制取得共享和一致性，这种途径在现有松耦合系统上不增加任何专用部件就可以实现。

（3）编译实现。自动将共享访问转换成同步和一致原语。它要求用户显式控制全局数据，当传送大量数据时或试图进行迁移时极其复杂。

### 3.5.3 对称多处理机结构

SMP 也称为共享存储多处理机，它与 MPP 最大的差别在于存储系统。SMP 有一个统一共享的 SM，而 MPP 则是每个处理机都拥有自己的 LM。

#### 1. 共享存储模型

共享存储方式有三种模型：均匀存储器存取（Uniform Memory Access, UMA）模型、非均匀存储器存取（Nonuniform Memory Access, NUMA）模型和只用高速缓存的存储器结构（Cache Only Memory Architecture, COMA）模型。这些模型的区别在于存储器和外设如何共享或分布。UMA 多处理机模型如图 3-6 所示，其物理存储器被所有处理机均匀共享。

NUMA 多处理机模型如图 3-8 所示。其访问时间随存储字的位置不同而变化，其 SM 物理上分布在所有处理机的 LM 上。所有 LM 的集合组成了全局地址空间，可被所有的处理机访问。处理机访问 LM 是比较快的，但访问属于另一台处理机的远程存储器则比较慢，因为通过互连网络会产生附加时延。

COMA 多处理机模型如图 3-9 所示。COMA 模型是 NUMA 机的一种特例，是将 NUMA 中的分布主存储器换成了高速缓存，在每个处理机节点上没有存储器层次结构，全部高速缓冲存储器组成了全局地址空间。远程高速缓存访问则需要借助于分布高速缓存目录进行。

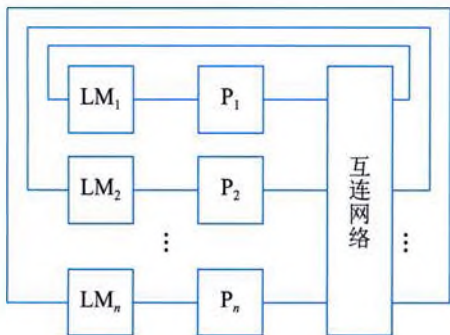


图 3-8 NUMA 多处理机模型

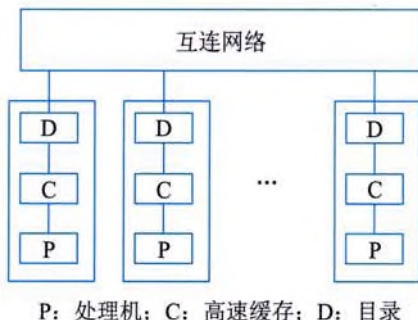


图 3-9 COMA 多处理机模型

共享存储器系统拥有统一寻址空间, 程序员不必参与数据分布和传输, 这种实现方式虽然简单, 但是阻碍了系统的扩展能力。于是, 有专家提出了一种称为 S2MP (Scalable Shared Memory Multi-Processing, 可扩展共享存储多处理机) 的并行计算机体系结构。S2MP 系统从性能和扩展能力两方面解决了 SMP 系统所存在的问题, 引进了复杂的存储子系统, 通过硬件 Cache 对系统的共享和私有数据都进行缓存, 以达到高性能的目标。

## 2. S2MP 的体系结构

S2MP 是一种共享存储的体系结构, 如图 3-10 所示。与 MPP 相比, 它支持简单的编程模型, 系统使用方便, 是对 SMP 系统在支持更高扩展能力方面的发展。共享存储系统降低了通信的额外开销, 因此, 系统也可以运行细粒度的应用。

从本质上来看, S2MP 是一种 NUMA 结构, 每个结点由处理机和存储器两部分组成, 存储器靠近处理机, 而不是集中在某个地方, 处理机可以访问 LM 获取数据。NUMA 结构可以降低平均访存时延, 并且随处理机数目的增加自动增加存储器带宽, 也就是说, 存储带宽是可扩展的。

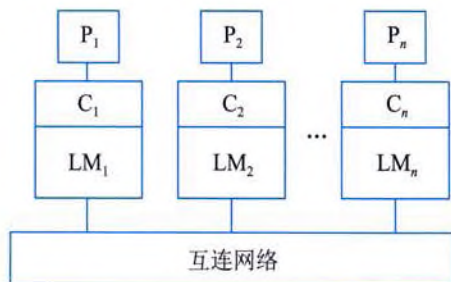


图 3-10 S2MP 体系结构示意图

### 3.5.4 互连网络

互连网络是用来连接一个计算机系统内的各处理部件、存储模块以及各种外设, 在系统软件控制下, 使各功能部件相互通信的硬件网络结构。目前, 互连网络已经成为并行处理系统的核心组成部分, 它对整个计算机系统的性能/价格比有着决定性的影响。

#### 1. 互连函数

为了反映不同互连网络的连接特性, 每种互连网络可用一组函数来描述。如果将互连网络的  $n$  个输入端和输出端分别用整数  $0, 1, \dots, n-1$  来表示, 则互连函数表示相互连接的输出端和输入端之间的一一对应关系。基本的互连函数主要有以下几种。

(1) 恒等置换。相同编号的输入端与输出端一一对应互连, 其表达式如下:

$$I(x_{n-1} \cdots x_k \cdots x_1 x_0) = x_{n-1} \cdots x_k \cdots x_1 x_0$$

(2) 交换置换。实现二进制地址编号中第 0 位位值不同的输入端和输出端之间的连接, 其表达式如下:

$$E(x_{n-1} \cdots x_k \cdots x_1 x_0) = x_{n-1} \cdots x_k \cdots \bar{x}_1 x_0$$

(3) 方体置换。实现二进制地址编号中第  $k$  位位值不同的输入端和输出端之间的连接, 其表达式如下:

$$C_k(x_{n-1} \cdots x_k \cdots x_1 x_0) = x_{n-1} \cdots \bar{x}_k \cdots x_1 x_0$$

(4) 均匀洗牌置换 (Shuffle)。将输入端二进制地址循环左移一位, 得到对应的输出端二进

制地址,其表达式如下:

$$S(x_{n-1}x_{n-2}\cdots x_1x_0) = x_{n-2}x_{n-3}\cdots x_1x_0x_{n-1}$$

(5) 蝶式置换。将输入端二进制地址的最高位和最低位互换位置,得到对应的输出端二进制地址,其表达式如下:

$$B(x_{n-1}x_{n-2}\cdots x_1x_0) = x_0x_{n-2}\cdots x_1x_{n-1}$$

(6) 位序颠倒置换。将输入端二进制地址的位序颠倒过来,得到对应的输出端二进制地址,其表达式如下:

$$P(x_{n-1}x_{n-2}\cdots x_1x_0) = x_0x_1\cdots x_{n-2}x_{n-1}$$

例如,编号为0,1,2,...,15的16个处理机,每个处理机均可用4位二进制编码来表示。如果采用单级互连网络连接,当互连函数为Cube<sub>3</sub>(k=3时的方体置换)时,则11号处理机连接到3号处理机上。因为11号处理机的编码为1011,它只能与编码为0011号的处理机相连接。如果采用Shuffle互连函数,则11号处理机的编码经过变换后为0111,即7号。也就是说,11号处理机与7号处理机连接。

## 2. 互连方式

在多处理机系统中,衡量互连网络性能好坏的主要因素是它的连接度、延时性、带宽、可靠性和成本,对于这些性能的度量,可以结合计算机网络和图论的知识来进行。例如,连接度的概念与图论中的度的概念是一致的,是指一个结点与其他结点的连接程度。如果一个结点直接连接的其他结点数越多,连接度就越高,表明连接性越好。延时性是指从一个结点传送信息到任何另一个结点所需的时间,通常可用结点间最大距离来表示。在设计互连网络时,应综合考虑通信工作方式、控制策略、交换方式和网络拓扑等因素,这也与计算机网络的设计是基本一致的。

在最典型和最常见的互连网络中,多处理机互连的方式主要有以下5种:

(1) 总线方式。这是最简单的方法,通过共享总线把各个处理机连接起来,再配备各处理机都可访问的全局存储器,每个处理机都能访问公共总线。该方式争用最严重。

(2) 交叉开关。该方式可以把争用现象降到最低程度,但连接复杂度最高。

(3) 开关枢纽。由仲裁单元和开关单元组成,前者完成冲突处理,后者完成连接。

(4) 多端口存储器。它是将交叉点仲裁逻辑移动到存储器去控制的方法。每个存储器模块有多个存取端口,由存储器负责分解多个处理机的冲突请求。

(5) 多级互连网络。MIMD和SIMD计算机都用多级互连网络,各种多级互连网络的区别就在于所用开关模块、控制方式和级间连接模式的不同。多级互连网络是总线方式和交叉开关两者的折中,其主要优点在于采用模块结构,因而扩展性好,然而时延随网络级数而上升。

## 3.6 操作系统

传统计算机系统资源分为硬件资源和软件资源。硬件资源包括中央处理机、存储器和输入/输出设备等物理设备;软件资源是以文件形式保存在存储器上的程序和数据等信息。现代计算

机系统资源管理范围已经扩展到感知、能源、通信资源和服务资源。本节主要介绍传统计算机系统资源管理。

### 3.6.1 操作系统概述

#### 1. 操作系统的定义及作用

操作系统(OS)能有效地组织和管理系统中的各种软/硬件资源,合理地组织计算机系统工作流程,控制程序的执行,并且向用户提供一个良好的工作环境和友好的接口。

操作系统是计算机系统的资源管理者,它含有对系统软/硬件资源实施管理的一组程序。操作系统有两个重要的作用:第一,通过资源管理提高计算机系统的效率;第二,改善人机界面,向用户提供友好的工作环境。

#### 2. 操作系统的特征与功能

操作系统是控制和管理计算机软/硬件资源,以尽可能合理、有效的方法组织多个用户共享多种资源的程序集合。它具有并发性、共享性、虚拟性和不确定性等特点,一般的操作系统都具有处理机管理、存储器管理、设备管理、文件管理和用户接口等5种主要功能。

(1) 处理机管理。负责对处理机的分配和运行实施有效的管理。在多道程序环境下,处理机的分配和运行是以进程为基本单位的。因此,处理机管理也称为进程管理。

(2) 存储器管理。存储器管理的主要任务是对内存进行分配、保护和扩充。

(3) 设备管理。设备管理应具有设备分配、设备传输控制和设备独立性等功能。其中,设备分配是指根据一定的原则对设备进行分配,为了使设备与主机并行工作,常需采用缓冲技术和虚拟技术;设备传输控制是指实现物理的输入/输出(Input/Output)操作,如启动设备、中断处理、结束处理等;设备独立性是指用户向系统申请的设备与实际操作的设备无关。

(4) 文件管理。负责对文件存储空间进行管理,包括存储空间的分配和回收、目录管理、文件操作管理和文件保护等功能。

(5) 用户界面。用户界面也称用户接口,是为了使用户能灵活、方便地使用计算机和系统功能,操作系统提供的一组友好的使用其功能的手段。用户接口包括程序接口和操作接口,通常,程序接口是面向开发人员的,操作接口是面向普通用户和管理者的。用户通过这些接口能方便地调用操作系统的功能,有效地组织作业和处理流程,并使系统能高效地运行。

#### 3. 操作系统的分类

通常,操作系统可分为批处理操作系统、分时操作系统、实时操作系统、网络操作系统、分布式操作系统和嵌入式操作系统等类型。本节主要介绍后5种。

##### 1) 分时操作系统

在分时操作系统中,一个计算机系统与多个终端设备连接。分时操作系统是将CPU的工作时间划分为许多很短的时间片,轮流为各个终端的用户服务。例如,一个带20个终端的分时系统,若每个用户每次分配一个50ms的时间片,则每隔1s即可为所有的用户服务一遍。因此,尽管各个终端上的作业是断续地运行的,但由于操作系统每次对用户程序都能做出及时的响应,

因此用户感觉整个系统均归其一人占用。

分时操作系统主要有4个特点：多路性、独立性、交互性和及时性。

## 2) 实时操作系统

实时是指计算机对于外来信息能够以足够快的速度进行处理，并在被控对象允许的时间范围内做出快速反应。实时操作系统（Real Time Operating System）是指对随机发生的外部事件应及时做出响应并对其进行处理。

实时操作系统分为实时控制系统和实时信息处理系统。实时控制系统主要用于生产过程的自动控制，例如，数据自动采集、武器控制、飞机自动驾驶和导弹的制导系统等；实时信息处理系统主要用于实时信息处理，例如，飞机订票系统、情报检索系统等。

实时操作系统与分时操作系统除了应用的环境不同外，主要有以下三点区别：

（1）系统的设计目标不同。分时操作系统设计成一个多用户的通用系统，交互能力强；而实时操作系统大多是专用系统。

（2）交互性的强弱不同。分时操作系统是多用户的通用系统，交互能力强；而实时操作系统是专用系统，仅允许操作并访问有限的专用程序，不能随便修改，且交互能力差。

（3）响应时间的敏感程度不同。分时操作系统是以用户能接受的等待时间为系统的设计依据，而实时操作系统是以被测物体所能接受的延迟为系统的设计依据。因此，实时操作系统对响应时间的敏感程度更强。

## 3) 网络操作系统

网络操作系统是使联网计算机能方便而有效地共享网络资源，为网络用户提供各种服务的软件和有关协议的集合。因此，网络操作系统的功能主要包括：高效、可靠的网络通信，对网络中共享资源（在LAN中有硬盘、打印机等）的有效管理，提供电子邮件、文件传输、共享硬盘和打印机等服务，网络安全管理，提供互操作能力。

网络操作系统可分为如下三类：

（1）集中模式。集中式网络操作系统是由分时操作系统加上网络功能演变而来的，系统的基本单元由一台主机和若干台与主机相连的终端构成，将多台主机连接起来形成了网络，信息的处理和控制在集中的。UNIX就是这类系统的典型例子。

（2）客户端/服务器模式。这是流行的网络工作模式，该种模式的网络可分为服务器和客户端。服务器是网络的控制中心，其任务是向客户端提供一种或多种服务，服务器可有多种类型，如提供文件/打印服务的文件服务器等。客户端是用于本地处理和访问服务器的站点，在客户端中包含了本地处理软件和访问服务器上服务程序的软件接口。

（3）对等（Peer-to-Peer）模式。在采用这种模式的操作系统网络中，各个站点是对等的。它既可作为客户端去访问其他站点，又可作为服务器向其他站点提供服务，在网络中既无服务处理中心，也无控制中心，或者说，网络的服务和控制功能分布在各个站点上。可见，该模式具有分布处理及分布控制的特征。

## 4) 分布式操作系统

分布式计算机系统是由多个分散的计算机经连接而成的计算机系统，系统中的计算机无主

次之分,任意两台计算机可以通过通信交换信息。通常,为分布式计算机系统配置的操作系统称为分布式操作系统。

分布式操作系统能直接对系统中的各类资源进行动态分配和调度,并能进行任务划分、信息传输协调工作,同时为用户提供一个统一的界面、标准的接口,用户通过这一界面实现所需要的操作和使用系统资源,使系统中若干台计算机相互协作完成共同的任务,有效地控制和协调诸任务的并行执行,并向系统提供统一、有效的接口的软件集合。

分布式操作系统是网络操作系统的更高级形式,它保持网络操作系统所拥有的全部功能,同时又有透明性、可靠性和高性能等特性。

#### 5) 嵌入式操作系统

嵌入式操作系统运行在嵌入式智能芯片环境中,对整个智能芯片以及它所操作、控制的各种部件装置等资源进行统一协调、处理、指挥和控制。其主要特点如下:

(1) 微型化。从性能和成本角度考虑,希望占用的资源和系统代码量少,如内存少、字长短、运行速度有限、能源少(用微小型电池)。

(2) 可定制。从减少成本和缩短研发周期考虑,要求嵌入式操作系统能运行在不同的微处理器平台上,能针对硬件变化进行结构与功能上的配置,以满足不同应用需要。

(3) 实时性。嵌入式操作系统主要应用于过程控制、数据采集、传输通信、多媒体信息及关键要害领域需要迅速响应的场合,所以对实时性要求较高。

(4) 可靠性。系统构件、模块和体系结构必须达到应有的可靠性,对关键要害应用还要提供容错和防故障措施。

(5) 易移植性。为了提高系统的易移植性,通常采用硬件抽象层(Hardware Abstraction Layer, HAL)和板级支撑包(Board Support Package, BSP)的设计技术。

嵌入式实时操作系统有很多,常见的有 VxWorks、 $\mu$ Clinux、PalmOS、Windows CE、 $\mu$ C/OS-II 和 eCos 等。

### 4. 互联网环境对操作系统的要求

互联网环境对 OS 有如下 5 个方面的要求:

(1) 自主配置和自适应协调。在互联网环境下,由于空间和时间上动态变化的系统组态,导致的问题主要包括如下三个方面:参与构成系统的节点数量在变化;系统的节点状态在变化;系统的需求不断变化。为了解决这些问题,Internet 平台对 OS 的要求是能自主配置和自适应协调。

(2) 支持跨网络的互连、互通和协作。对于当前及未来 Internet 环境,需要考虑的问题包括:有线与无线通信能力的结合;固定和移动节点的动态结合;数据资源与物理传感资源的结合;人机交互界面的多元化(如语音、影像以及各种物理、化学和生物信息的交互)。因此,面临计算单元的分布、动态、异构,以及协同和演化特性等问题。

(3) 管理和协调系统资源的共享。随着 OS 资源管理范围从传统的计算资源(处理器、存储器、外部设备)、数据资源、软件资源等,扩大到感知、能源、通信资源和服务资源,其面临如何管理和协调系统资源的共享。每个参与节点的能源消耗和储备成为资源管理的重要内容,

高层服务资源也成为 OS 管理对象,这是对系统之上应用层的扩展,例如,Internet 搜索和桌面搜索功能,已成为 OS 的基本功能之一。

(4) 支持功能、性能和可信性指标动态演化。对运算结果的控制要综合考虑能源消耗、运算精度以及快速处理等三个方面的要求之后,进行折中处理。

(5) 可信赖操作系统。开放环境所要求的整体化、多层次的可信性。构造可信赖系统包括私密性、防卫性、可靠性、安全性以及可用性。人们对信息安全给予空前的关注,这就要求 OS 能够支持更高强度的安全性能。

总之,软硬件之间的异构性促进了软件技术的发展。为屏蔽计算机硬件的异构性,发展了操作系统;为屏蔽操作系统和编程语言的异构性,发展了支撑软件和中间件;为了弥补应用软件与开发平台之间的距离,发展了软件框架、设计模式等技术。

### 3.6.2 进程管理

#### 1. 程序与进程

##### 1) 前驱图与前驱关系

前趋图是一个有向无循环图,由结点和有向边组成,结点代表各程序段的操作,而结点间的有向边表示两个程序段操作之间存在的前趋关系( $\rightarrow$ )。程序段  $P_i$  和  $P_j$  的前趋关系表示成  $P_i \rightarrow P_j$ ,其中, $P_i$  是  $P_j$  的直接前趋, $P_j$  是  $P_i$  的直接后继,其含义是  $P_i$  执行结束后  $P_j$  才能执行。程序顺序执行时的主要特征包括顺序性、封闭性和可再现性。

前驱关系“ $\rightarrow$ ”可形式化地记为  $\rightarrow = \{ \langle P_i, P_j \rangle \mid P_i \text{ 必须在 } P_j \text{ 执行前完成} \}$

**【例 1】**某系统中有一个 CPU、一台输入设备和一台输出设备,每个作业具有 3 个程序段,输入  $I_i$ 、计算  $C_i$  和输出  $P_i$  ( $i=1,2,3$ )。三个作业的各程序段并发执行的前驱图如图 3-11 所示。请给出该图的前驱关系。

**【解】**从图 3-11 中可以看出, $I_2$  与  $C_1$  并行执行; $I_3$ 、 $C_2$  与  $P_1$  并行执行; $C_3$  与  $P_2$  并行执行。其中, $I_2$ 、 $I_3$  受到  $I_1$  的间接制约, $C_2$ 、 $C_3$  受到  $C_1$  的间接制约, $P_2$ 、 $P_3$  受到  $P_1$  的间接制约,而  $C_1$ 、 $P_1$  受到  $I_1$  的直接制约,等等。

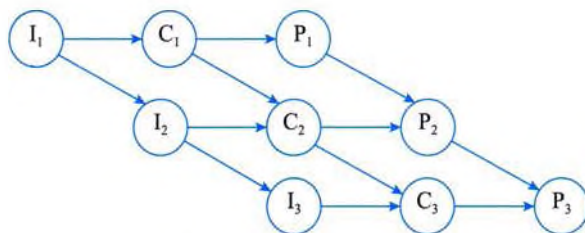


图 3-11 程序并发执行的前驱图

经上分析,图 3-11 中的前驱关系可记为:

$$\rightarrow = \{ \langle I_1, C_1 \rangle, \langle I_1, I_2 \rangle, \langle I_2, C_2 \rangle, \langle I_2, I_3 \rangle, \langle I_3, C_3 \rangle, \langle C_1, P_1 \rangle, \langle C_1, C_2 \rangle, \langle C_2, P_2 \rangle, \langle C_2, C_3 \rangle, \langle C_3, P_3 \rangle, \langle P_1, P_2 \rangle, \langle P_2, P_3 \rangle \}$$

或者也可以记为如下形式:

$$\rightarrow = \{I_1 \rightarrow C_1, I_1 \rightarrow I_2, I_2 \rightarrow C_2, I_2 \rightarrow I_3, I_3 \rightarrow C_3, C_1 \rightarrow P_1, C_1 \rightarrow C_2, C_2 \rightarrow P_2, C_2 \rightarrow C_3, \\ C_3 \rightarrow P_3, P_1 \rightarrow P_2, P_2 \rightarrow P_3\}$$

程序并发执行的特征包括三方面: 第一, 失去了程序的封闭性; 第二, 程序和机器执行程序的活动不再一一对应; 第三, 并发程序间的相互制约性。

## 2) 进程的组成

进程是程序的一次执行, 该程序可以和其他程序并发执行。进程通常是由进程控制块 (Process Control Block, PCB)、程序和数据组成的。

(1) PCB。PCB 是进程存在的唯一标志。

(2) 程序。程序部分描述了进程需要完成的功能。假如一个程序能被多个进程同时共享执行, 那么这一部分就应该以可再入 (纯) 码的形式编制, 它是程序执行时不可修改的部分。

(3) 数据。数据部分包括程序执行时所需的数据及工作区。该部分只能为一个进程专用, 是进程的可修改部分。

## 2. 进程的状态与状态模型图

### 1) 三态模型

在多道程序系统中, 进程在处理器上交替运行, 状态也不不断地发生变化, 因此进程一般有三种基本状态: 运行、就绪和阻塞。进程的三态模型如图 3-12 所示。

(1) 运行。当一个进程在处理器上运行时, 则称该进程处于运行状态。显然, 对于单处理机系统, 处于运行状态的进程只有一个。

(2) 就绪。一个进程获得了除处理机外的一切所需资源, 一旦得到处理机即可运行, 则称此进程处于就绪状态。

(3) 阻塞。阻塞也称等待或睡眠状态, 一个进程正在等待某一事件发生 (例如, 请求 I/O、等待 I/O 完成等) 而暂时停止运行, 这时即使把处理机分配给进程也无法运行, 故称该进程处于阻塞状态。

### 2) 五态模型

事实上, 对于一个实际的系统, 进程的状态及其转换更复杂。例如, 引入新建态和终止态, 构成了进程的五态模型, 如图 3-13 所示。

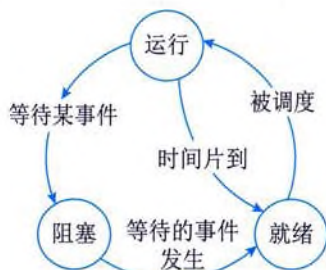


图 3-12 进程的三态模型

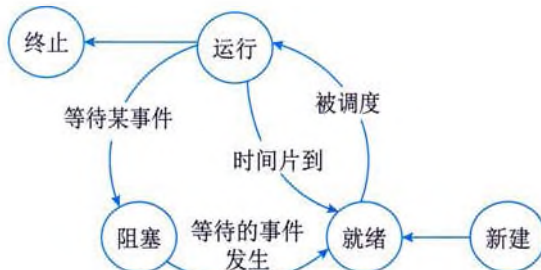


图 3-13 进程的五态模型

在图 3-13 中，新建态对应于进程刚刚被创建时没有被提交的状态，并等待系统完成创建进程的所有必要信息。因为创建进程时分为两个阶段，第一个阶段为一个新进程创建必要的管理信息，第二个阶段让该进程进入就绪状态。由于有了新建态操作系统，往往可以根据系统的性能和主存容量的限制推迟新建态进程的提交。

类似地，进程的终止也可分为两个阶段，第一个阶段等待操作系统进行善后处理，第二个阶段释放主存。

### 3. 信号量与 PV 操作

荷兰学者 Dijkstra 于 1965 年提出的信号量机制是一种有效的进程同步与互斥工具。目前，信号量机制有了很大的发展，主要有整型信号量、记录型信号量和信号量集机制。

#### 1) 整型信号量与 PV 操作

信号量是一个整型变量，根据控制对象的不同被赋予不同的值。信号量分为如下两类：

- (1) 公用信号量。实现进程间的互斥，初值为 1 或资源的数目。
- (2) 私用信号量。实现进程间的同步，初值为 0 或某个正整数。

信号量  $S$  的物理意义： $S \geq 0$  表示某资源的可用数；若  $S < 0$ ，则其绝对值表示阻塞队列中等待该资源的进程数。

对于系统中的每个进程，其工作的正确与否不仅取决于它自身的正确性，而且与它在执行中能否与其他相关进程正确地实施同步与互斥有关。PV 操作是实现进程同步与互斥的常用方法。P 操作和 V 操作是低级通信原语，在执行期间不可分割。其中，P 操作表示申请一个资源，V 操作表示释放一个资源。

P 操作的定义： $S := S - 1$ ，若  $S \geq 0$ ，则执行 P 操作的进程继续执行；若  $S < 0$ ，则置该进程为阻塞状态（因为无可用资源），并将其插入阻塞队列。P 操作可用如下过程表示。

```
Procedure P(Var S:Semaphore); // Semaphore 表示所定义的变量是信号量
Begin
    S:=S-1;
    If S<0 then W(S)           {执行 P 操作的进程插入等待队列}
End;
```

V 操作的定义： $S := S + 1$ ，若  $S > 0$ ，则执行 V 操作的进程继续执行；若  $S \leq 0$ ，则从阻塞状态唤醒一个进程，并将其插入就绪队列，然后执行 V 操作的进程继续。

V 操作可用如下过程表示。

```
Procedure V(Var S:Semaphore);
Begin
    S:=S+1;
    If S≤0 then R(S)           {从阻塞队列中唤醒一个进程}
End;
```

#### 2) 利用 PV 操作实现进程的互斥

令信号量  $\text{mutex}$  的初值为 1，当进入临界区时执行 P 操作，退出临界区时执行 V 操作。这

样, 利用 PV 操作实现进程互斥的代码段如下:

P(mutex)

临界区

V(mutex)

### 3) 利用 PV 操作实现进程的同步

进程的同步是由于进程间合作引起的相互制约的问题, 要实现进程的同步可用一个信号量与消息联系起来, 当信号量的值为 0 时表示希望的消息未产生, 当信号量的值为非 0 时表示希望的消息已经存在。假定用信号量  $S$  表示某条消息, 进程可以通过调用 P 操作测试消息是否到达, 调用 V 操作通知消息已准备好。最典型的同步问题是缓冲区的生产者和消费者的同步问题。

**【例 2】**一个生产者和一个消费者, 缓冲区中可存放  $n$  件产品, 生产者不断地生产产品, 消费者不断地消费产品, 如何用 PV 操作实现生产者和消费者的同步。

**【解】**本题可以通过设置 3 个信号量  $S$ 、 $S_1$  和  $S_2$  来解决。其中:

$S$  是一个互斥信号量, 初值为 1。由于缓冲区是一个互斥资源, 所以需要进行互斥控制。

$S_1$  是一个同步信号量, 初值为  $n$ , 表示缓冲区的空闲单元数, 即是否可以将产品放入缓冲区。

$S_2$  是一个同步信号量, 表示缓冲区是否存有产品, 初值为 0。

综上所述, 一个生产者、一个消费者的同步过程如图 3-14 所示。

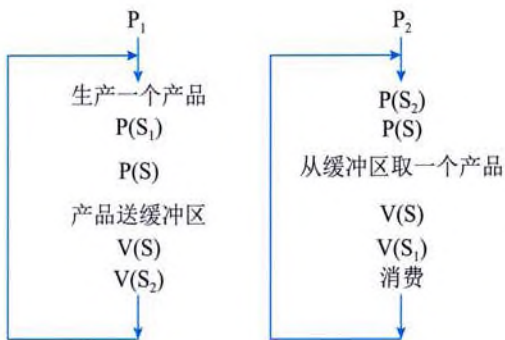


图 3-14  $n$  个缓冲区的同步举例

## 4. 高级通信原语

进程间通信是指进程之间的信息交换, 少则一个信息, 多则成千上万个信息。根据交换信息量的多少和效率的高低, 进程通信的方式分为低级方式和高级方式。PV 操作属于低级通信方式, 若用 PV 操作实现进程间通信, 则存在如下问题:

(1) 编程难度大。通信对用户不透明, 即要用户利用低级通信工具实现进程间的同步与互斥。而且, PV 操作使用不当容易引起死锁。

(2) 效率低。在上例中, 生产者每次只能向缓冲区送一个产品, 消费者每次只能从缓冲区取一个产品。

为了提高信号通信的效率, 传递大量数据, 降低程序编制的复杂度, 系统引入了高级通信方式。高级通信方式主要分为共享存储模式、消息传递模式和管道通信。

(1) 共享存储模式。相互通信的进程可以共享某些数据结构 (或存储区) 实现进程之间的通信。

(2) 消息传递模式。进程间的数据交换以消息为单位, 程序员直接利用系统提供的一组通信命令 (原语) 来实现通信, 如 Send(A)、Receive(A)。

(3) 管道通信。所谓管道，是指用于连接一个读进程和一个写进程，以实现它们之间通信的共享文件（pipe 文件）。向管道（共享文件）提供输入的发送进程（即写进程），以字符流的形式将大量的数据送入管道；而接收进程可从管道接收大量的数据。由于它们通信时采用管道，所以称为管道通信。

## 5. 产生死锁的原因及处理策略

### 1) 产生死锁的原因

死锁是指两个以上的进程互相都要求使用对方已经占有的资源而导致无法继续运行下去的现象。产生死锁的主要原因是，系统缺少足够的资源供进程使用，对互斥资源的共享与并发执行的顺序不当，以及资源分配不当。产生死锁的4个必要条件如下：

- (1) 互斥条件。任一时刻只允许一个进程使用资源。
- (2) 不剥夺条件。进程已经占用的资源，不会被强制剥夺。
- (3) 请求与保持条件。进程在请求其余资源时，不主动释放已经占有的资源。
- (4) 环路条件。环路中每一条边是进程在请求另一个进程已经占有的资源。

### 2) 死锁的处理策略

死锁的处理策略主要有4种：鸵鸟策略（即不理睬策略）、预防策略、避免策略和检测与解除死锁。

(1) 死锁预防。死锁预防是采用某种策略限制并发进程对资源的请求，破坏死锁产生的4个必要条件之一，使系统在任何时刻都不满足死锁的必要条件。预防死锁的两种策略如下：

- 预先静态分配法。破坏了“不可剥夺条件”，预先分配所需资源，保证不等待资源。该方法的问题是降低了对资源的利用率，降低进程的并发程度；有时可能无法预先知道所需资源。
- 资源有序分配法。破坏了“环路条件”，把资源分类按顺序排列，保证不形成环路。该方法存在的问题是限制进程对资源的请求；因资源排序占用系统开销。

(2) 死锁避免。死锁避免是设法破坏产生死锁的4个必要条件之一，严格防止死锁的产生。死锁避免则不那么严格地限制产生死锁的必要条件。最著名的死锁避免算法是 Dijkstra 提出的银行家算法，死锁避免算法需要很大的系统开销。

银行家算法对于进程发出的每一个系统可以满足的资源请求命令加以检测，如果发现分配资源后系统进入不安全状态，则不予分配；若分配资源后系统仍处于安全状态，则实施分配。

与死锁预防策略相比，它提高了资源的利用率，但检测分配资源后系统是否安全增加了系统开销。安全状态是指系统能按某种顺序，如  $\langle P_1, P_2, \dots, P_n \rangle$  来为每个进程分配其所需资源，直到最大需求，使每个进程都可顺序完成，则称  $\langle P_1, P_2, \dots, P_n \rangle$  序列为安全序列；若系统不存在这样一个安全序列，则称系统处于不安全状态。

### 3) 死锁检测与解除

解决死锁的另一条途径是使用死锁检测方法，这种方法对资源的分配不加限制，即允许死

锁产生。但系统定时地运行一个死锁检测程序,判断系统是否发生死锁,若检测到有死锁,则设法加以解除。死锁解除通常采用资源剥夺法和撤销进程法。资源剥夺法是从一些进程那里强行剥夺足够数量的资源分配给死锁进程;撤销进程法是根据某种策略逐个地撤销死锁进程,直到解除死锁为止。

## 6. 线程

传统的进程有两个基本属性:可拥有资源的独立单位;可独立调度和分配的基本单位。引入线程的原因是进程在创建、撤销和切换中,系统必须为之付出较大的时空开销,故在系统中设置的进程数目不宜过多,进程切换的频率不宜太高。

引入线程后,将传统进程的两个基本属性分开,线程作为调度和分配的基本单位,进程作为独立分配资源的单位。用户可以通过创建线程来完成任务,以减少程序并发执行时付出的时空开销。例如,在文件服务进程中可设置多个服务线程,当一个线程受阻时,第二个线程可以继续运行,当第二个线程受阻时,第三个线程可以继续运行……从而显著地提高文件系统的服务质量及系统的吞吐量。这样,对于拥有资源的基本单位,不用频繁地切换,进一步提高了系统中各程序的并发程度。

需要说明的是,线程是进程中的一个实体,是被系统独立分配和调度的基本单位。线程基本上不拥有资源,只拥有一点运行中必不可少的资源(如程序计数器、一组寄存器和栈),它可与同属一个进程的其他线程共享进程所拥有的全部资源。

线程也具有就绪、运行和阻塞3种基本状态。由于线程具有许多传统进程所具有的特性,故称为“轻型进程(Light-Weight Process)”。线程可创建另一个线程,同一个进程中的多个线程可并发执行。线程分为用户级线程(User-Level Threads)和内核支持线程(Kernel-Supported Threads)两类。用户级线程不依赖于内核,该类线程的创建、撤销和切换都不利用系统调用来实现;内核支持线程依赖于内核,即无论是在用户进程中的线程,还是在系统中的线程,它们的创建、撤销和切换都利用系统调用来实现。

### 3.6.3 存储器管理

存储器管理的对象是主存存储器,简称为主存或内存。存储器是计算机系统中的关键性资源,是存放各种信息的主要场所。如何对存储器实施有效的管理,不仅直接影响存储器的利用率,而且还对系统性能有很大的影响。存储器管理的主要功能包括主存空间的分配和回收、提高主存的利用率、扩充主存、对主存信息实现有效保护。

存储器管理的主要目的是解决多个用户使用主存的问题,其存储管理方案主要包括分区存储管理、分页存储管理、分段存储管理、段页式存储管理以及虚拟存储管理。

#### 1. 分区存储管理

分区存储管理是早期的存储管理方案,其基本思想是把主存的用户区划分成若干个区域,每个区域分配给一个用户作业使用,并限定它们只能在自己的区域中运行。

按划分方式不同,分区存储管理可分为固定分区、可变分区和可重定位分区。其中,固定分区是一种静态分区方式;可变分区是一种动态分区方式;可重定位分区的基本思想是移动所

有已分配好的分区，使之成为连续区域，以减少内存中的碎片。

## 2. 分页存储管理

在分页存储管理中，地址结构由两部分组成：页号  $P$  和页内地址  $W$ 。系统为每个进程建立了一张页面映射表（简称页表），记录进程的每个页面对应的物理块号。

分页存储管理的地址变换机构如图 3-15 所示，其任务是利用页表把用户程序中的逻辑地址变换成主存中的物理地址。

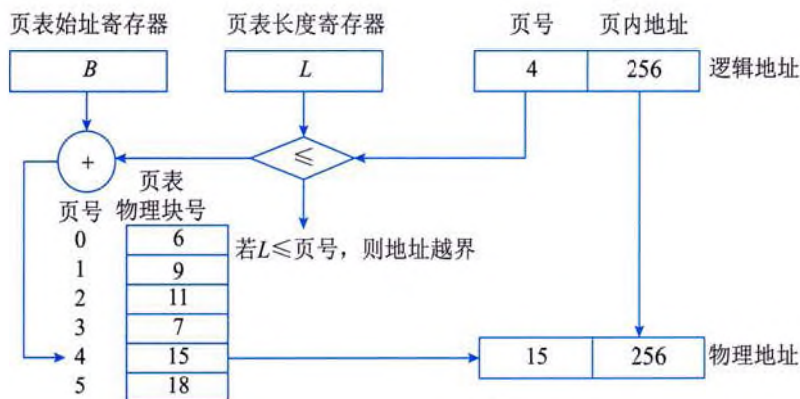


图 3-15 分页存储管理的地址变换机构

在进行地址变换时，系统将页号（页号从 0 开始）与页表长度寄存器中的  $L$  进行比较，若页号大于等于  $L$ ，则访问越界，产生越界中断；若未越界，则根据页表寄存器中的页表起始地址和页号计算出该页在页表项中的位置，得到该页的物理块号，将此物理块号装入物理地址寄存器中。与此同时，将有效地址（逻辑地址）寄存器中的页内地址直接装入物理地址寄存器的块内地址字段中，这样便完成了从逻辑地址到物理地址的变换。

进程在执行时，系统通过查找页表就可以找到每页所对应的物理块号。例如，图 3-15 中逻辑页号为 4，查找页表可得该页的物理块号为 15，与页内地址 256 拼接得到物理地址 15256。可见，页表的作用是实现从页号到物理块号的地址映射。

## 3. 分段存储管理

在分段存储管理方式中，作业的地址空间被划分为若干个段，每个段是一组完整的逻辑信息，例如有主程序段、子程序段、数据段及堆栈段等，每个段都有自己的名字，都是从 0 开始编址的一段连续的地址空间，各段的长度是不等的。分段系统的逻辑地址结构由段号（名）和段内地址两部分组成。

在分段存储管理系统中，为每个段分配一个连续的分区，而进程中的各个段可以离散地分配到主存的不同分区中。在系统中为每个进程建立一张段映射表，简称为段表。每个段在表中占有一个表项，在其中记录了该段在主存中的起始地址（又称为基址）和段的长度。进程在执行时，通过查段表来找到每个段所对应的主存区。可见，段表实现了从逻辑段到物理主存区的映射。分段存储管理的地址变换机构如图 3-16 所示。

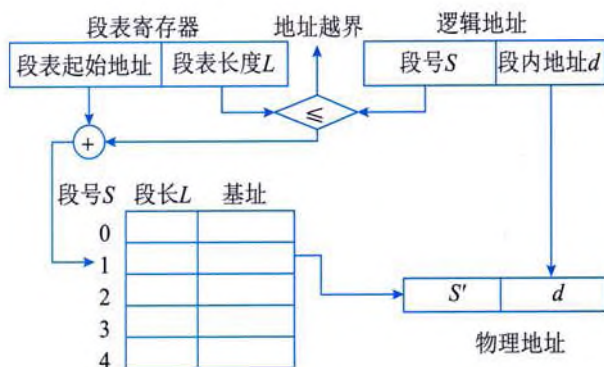


图 3-16 分段存储管理的地址变换机构

为了实现从逻辑地址到物理地址的变换功能，系统中设置了段表寄存器，用于存放段表起始地址和段表长度。在进行地址变换时，系统将逻辑地址中的段号  $S$  与段表长度  $L$  进行比较。若  $S \geq L$ ，表示段号太大，访问越界，产生越界中断信号；否则，根据段表的起始地址和该段的段号，计算出该段对应段表项的位置，从中读出该段在主存中的起始地址，然后再检查段内地址  $d$  是否超过该段的段长  $SL$ 。若超过，即  $d \geq SL$ ，同样发出越界中断信号；否则，将该段的基址  $S'$  与段内地址  $d$  相加，得到要访问的主存物理地址。

#### 4. 段页式存储管理

段页式存储管理的基本原理是先将整个主存划分成大小相等的存储块（页框），将用户程序按程序的逻辑关系分为若干个段，并为每个段赋予一个段名，再将每个段划分成若干页，以页框为单位离散分配。段页式存储管理的地址变换机构如图 3-17 所示。

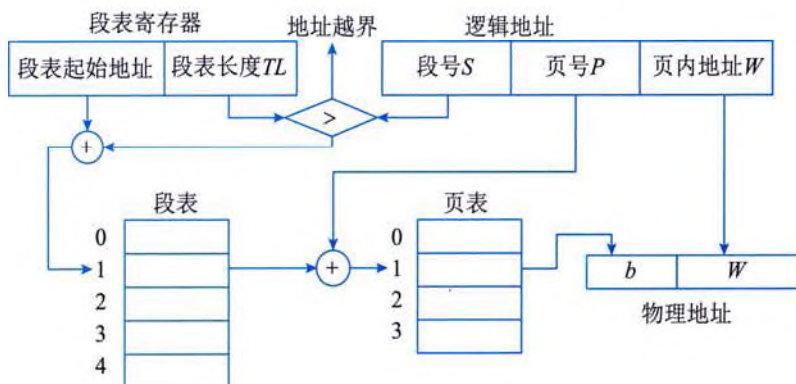


图 3-17 段页式存储管理的地址变换机构

地址结构由段号、段内页号和页内地址三部分组成。为了实现从逻辑地址到物理地址的变换，系统中必须同时配置段表和页表，段表中的内容是页表的起始地址和页表长度。

在段页式存储管理中，逻辑地址到物理地址的变换过程如下：

- (1) 根据段号  $S$  查段表，得到页表的起始地址；
- (2) 根据页号  $P$  查页表，得到物理块号  $b$ ；

(3) 将物理块号  $b$  拼页内地址  $W$  得到物理地址。

## 5. 虚拟存储管理

在前面介绍的存储管理方案中，必须为每个作业分配足够的空间，以便装入全部信息。当主存空间不能满足作业要求时，作业无法装入主存执行。

如果一个作业只部分装入主存便可开始启动运行，其余部分暂时留在磁盘上，在需要时再装入主存，这样可以有效地利用主存空间。从用户角度看，该系统所具有的主存容量将比实际主存容量大得多，人们把这样的存储器称为虚拟存储器。虚拟存储器是为了扩大主存容量而采用的一种设计方法，其容量是由计算机的地址结构决定的。

### 1) 程序的局限性原理

早在 1968 年 P. Denning 就指出，程序在执行时将呈现出局部性规律，即在一段时间内，程序的执行仅局限于某个部分。相应地，它所访问的存储空间也局限于某个区域内。程序的局限性表现在时间局限性和空间局限性两个方面。

(1) 时间局限性是指如果程序中的某条指令一旦执行，则不久的将来该指令可能再次被执行；如果某个存储单元被访问，则不久以后该存储单元可能再次被访问。产生时间局限性的典型原因是，在程序中存在大量的循环操作。

(2) 空间局限性是指一旦程序访问了某个存储单元，则在不久的将来，其附近的存储单元也最有可能被访问，即程序在一段时间内所访问的地址可能集中在一定的范围内，其典型原因是程序的顺序执行。

### 2) 工作集理论

事实上，程序在运行中所产生的缺页情况会影响程序的运行速度及系统性能，而缺页率的高低又与每个进程所占用的物理块数目有关。但是，究竟应该为每个进程分配多少个物理块才能把缺页率保持在一个合理的水平上是无法预测的，否则会因为进程频繁地从辅存请求页面而出现“颠簸”（也称抖动）现象。为了解决这一问题，引入了工作集理论。

工作集的理论是在 1968 年由 P. Denning 提出的，他认为，虽然程序只需有少量的几页在主存就可以运行，但为了使程序能够有效地运行，较少地产生缺页，必须使程序的工作集驻留在主存中。把某进程在时间  $t$  的工作集记为  $w(t, \Delta)$ ，变量  $\Delta$  称为工作集窗口尺寸。正确地选择工作集窗口 ( $\Delta$ ) 的大小，对存储器的有效利用和系统吞吐量的提高都将产生重大的影响。可见，工作集就是指在某段时间间隔 ( $\Delta$ ) 里，进程实际要访问的页面的集合。

需要注意的是，程序在运行时对页面的访问是不均匀的，如果能够预知程序在某段时间间隔内要访问哪些页面，并能将它们提前调入主存，将会大大地降低缺页率，从而减少置换工作，提高 CPU 的利用率。当每个工作集都已达到最小值时，虚存管理程序跟踪进程的缺页数量，根据主存中自由页面的数量可以适当增加其工作集的大小。

### 3) 虚拟存储器的实现

虚拟存储器是具有请求调入功能和置换功能，仅把作业的一部分装入主存便可运行作业的存储器系统，是能从逻辑上对主存容量进行扩充的一种虚拟的存储器系统。其逻辑容量由主存和外存容量之和以及 CPU 可寻址的范围来决定，其运行速度接近于主存速度，成本也有所下

降。可见,虚拟存储技术是一种性能非常优越的存储器管理技术,故被广泛地应用于大、中、小型机器和微型机中。虚拟存储器的实现主要有如下3种方式:

(1) 请求分页系统。该系统是在分页系统的基础上增加了请求调页功能和页面置换功能所形成的页式虚拟存储系统。它允许只装入若干页的用户程序和数据(而非全部程序)就可以启动运行,以后再通过调页功能和页面置换功能陆续把将要使用的页面调入主存,同时把暂不运行的页面置换到外存上,置换时以页面为单位。

(2) 请求分段系统。该系统是在分段系统的基础上增加了请求调段功能和分段置换功能所形成的段式虚拟存储系统。它允许只装入若干段的用户程序和数据就可以启动运行,之后再通过调段功能和分段置换功能将不运行的段调出,同时调入将要运行的段。置换时以段为单位。

(3) 请求段页式系统。该系统是在段页式系统的基础上增加了请求调页功能和页面置换功能所形成的段页式虚拟存储系统。

#### 4) 页面置换算法

请求分页是在纯分页系统的基础上增加了请求调页功能、页面置换功能所形成的页式虚拟存储系统,它是目前常用的一种虚拟存储器的方式。在进程运行过程中,如果发生缺页,此时主存中又无空闲块时,为了保证进程能正常运行,必须从主存中调出一页程序或数据送到磁盘的对换区。但究竟将哪个页面调出,需要根据一定的页面置换算法来确定。置换算法的好坏将直接影响系统的性能,不适当的算法可能会导致系统发生“抖动”(Thrashing),即刚被换出的页很快又被访问,需重新调入,导致系统频繁地更换页面,以至于一个进程在运行中把大部分时间花费在完成页面置换的工作上,这种现象称为系统发生了“抖动”(也称颠簸)。请求分页系统的核心问题是选择合适的页面置换算法。常用的页面置换算法如下所述:

(1) 最佳(Optimal)置换算法。这是一种理想化的算法,即选择那些永不使用的,或者是在最长时间不再被访问的页面置换出去。这种方法性能最好,但实际上难以实现,并且要确定哪一个页面是未来最长时间不再被访问的是很难的,所以该算法通常用来评价其他算法。

(2) 先进先出(FIFO)置换算法。该算法总是淘汰最先进入主存的页面,即选择在主存中驻留时间最久的页面予以淘汰。该算法实现简单,只需把一个进程调入主存的页面,按先后次序链接成一个队列,并设置一个指针即可。FIFO是一种最直观、性能最差的算法,并且有Belady异常现象。所谓Belady现象,是指如果对一个进程未分配它所要求的全部页面,有时就会出现分配的页面数增多但缺页率反而提高的异常现象。

(3) 最近最少使用(Least Recently Used, LRU)置换算法。LRU算法是选择最近最少使用的页面予以淘汰,系统在每个页面设置一个访问字段,用于记录这个页面自上次被访问以来所经历的时间 $T$ ,当要淘汰一个页面时,选择 $T$ 最大的页面,但在实现时需要硬件的支持(寄存器或栈)。

(4) 最近未用(Not Used Recently, NUR)置换算法。NUR算法将最近一段时间未引用过的页面换出,这是一种LRU的近似算法。该算法为每个页面设置一位访问位,将主存中的所有页面都通过链接指针链成一个循环队列。当某页被访问时,其访问位置1。在选择一页淘汰时,检查其访问位,如果是0,则选择该页换出;若为1,则重新置为0,暂不换出该页,在循环

队列中检查下一个页面，直到访问位为 0 的页面为止。由于该算法只有一位访问位，只能用它表示该页是否已经使用过，而置换时是将未使用过的页面换出去，所以把该算法称为最近未用算法。

### 3.6.4 设备管理

在计算机系统中，将负责管理设备和输入/输出的机构称为 I/O 系统。I/O 系统由设备、控制器、通道（具有通道的计算机系统）、总线和 I/O 软件组成。

设备管理的目标主要是提高设备的利用率，为用户提供方便、统一的界面。提高设备的利用率，就是提高 CPU 与 I/O 设备之间的并行操作程度。在设备管理中，主要利用的技术有中断技术、DMA 技术、通道技术和缓冲技术。

#### 1. I/O 软件

I/O 设备管理软件一般分为 4 层：中断处理程序、设备驱动程序、与设备无关的系统软件 and 用户级软件。至于一些具体分层时细节上的处理，是依赖于系统的，没有严格的划分，只要有利于设备独立这一目标，就可以为了提高效率设计不同的层次结构。

I/O 软件的所有层次及每一层的主要功能如图 3-18 所示，图中的箭头给出了 I/O 部分的控制流。



图 3-18 I/O 系统的层次结构及各层的主要功能

例如，读一个硬盘上的文件。用户程序需要通过操作系统实现这一操作，其工作步骤为：与设备无关软件检查高速缓存中是否有要读的数据块，若没有，则调用设备驱动程序，向 I/O 硬件发出一个请求；用户进程阻塞并等待磁盘操作的完成；当磁盘操作完成时，硬件产生一个中断，转入中断处理程序；中断处理程序检查中断的原因，认识到这时磁盘读取操作已经完成，于是唤醒用户进程取回从磁盘读取的信息，从而结束此次 I/O 请求；用户进程在得到了所需的硬盘文件内容之后，继续运行。

#### 2. 设备管理采用的关键技术

设备管理采用的关键技术包括通道技术、DMA 技术、缓冲技术和 SPOOLing 技术。通道技术和 DMA 技术参见 3.3.1 小节。本小节介绍 SPOOLing 技术。

SPOOLing (Simultaneous Peripheral Operations On Line) 也称外围设备联机操作。实际上，SPOOLing 技术是用一类物理设备模拟另一类物理设备，使得独占使用的设备变成多台虚拟设备的一种技术。SPOOLing 系统的组成和结构如图 3-19 所示。

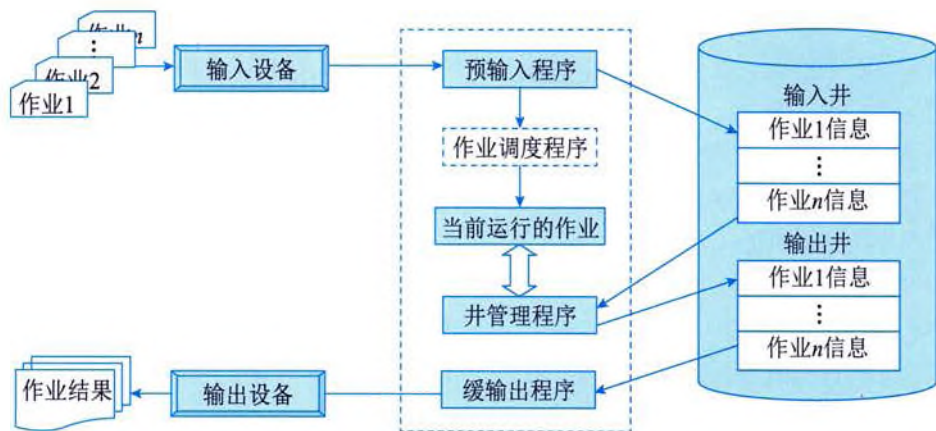


图 3-19 SPOOLing 系统的组成和结构

SPOOLing 系统是由“预输入程序”“缓输出程序”“井管理程序”以及输入和输出井组成的。其中，输入井和输出井是为了存放从输入设备输入的信息以及作业执行的结果，输入井和输出井是系统在辅助存储器上开辟的存储区域。

SPOOLing 系统的工作过程是：操作系统初启后激活 SPOOLing 预输入程序，使它处于捕获输入请求的状态，一旦有输入请求消息，SPOOLing 输入程序立即得到执行，把装在输入设备上的作业输入到硬盘的输入井中并填写好作业表，以便在作业执行中要求输入信息时可以随时找到它们的存放位置。当作业需要输出数据时，可以先将数据送到输出井，当输出设备空闲时，由 SPOOLing 输出程序把硬盘上输出井的数据送到慢速的输出设备上。

SPOOLing 系统中拥有一张作业表，用来登记进入系统的所有作业的作业名、状态和预输入表位置等信息。每个用户作业拥有一张预输入表来登记该作业的各个文件的情况，包括设备类、信息长度及存放位置等。

输入井中的作业有如下 4 种状态：

- (1) 提交状态。作业的信息正从输入设备上预输入。
- (2) 后备状态。作业预输入结束但未被选中执行。
- (3) 执行状态。作业已被选中运行，在运行过程中，它可从输入井中读取数据信息，也可向输出井写信息。
- (4) 完成状态。作业已经撤离，该作业的执行结果等待缓输出。

### 3. 磁盘调度

磁盘是可被多个进程共享的设备。当有多个进程请求访问磁盘时，为了保证信息的安全，系统在每一时刻只允许一个进程启动磁盘进行 I/O 操作，其余的进程只能等待。因此，操作系统应采用一种适当的调度算法，使各进程对磁盘的平均访问（主要是寻道）时间最小。磁盘调度分为移臂调度和旋转调度两类，并且是先进行移臂调度，然后进行旋转调度。由于访问磁盘最耗时的是寻道时间，因此，磁盘调度的目标是使磁盘的平均寻道时间最少。

常用的磁盘调度算法如下：

- (1) 先来先服务（First-Come First-Served, FCFS）。这是最简单的磁盘调度算法，它根据进

程请求访问磁盘的先后次序进行调度。此算法的优点是公平、简单，且每个进程的请求都能依次得到处理，不会出现某进程的请求长期得不到满足的情况。但此算法由于未对寻道进行优化，致使平均寻道时间可能较长。

(2) 最短寻道时间优先 (Shortest Seek Time First, SSTF)。该算法选择这样的进程，其要求访问的磁道与当前磁头所在的磁道距离最近，使得每次的寻道时间最短。但这种调度算法不能保证平均寻道时间最短。

(3) 扫描算法 (SCAN)。扫描算法不仅考虑到要访问的磁道与当前磁道的距离，更优先考虑的是磁头的当前移动方向。例如，当磁头正在由里向外移动时，SCAN 所选择的下一个访问对象应是其要访问的磁道既在当前磁道之外，又是距离最近的。这样由里向外地访问，直到再无更外的磁道需要访问时才将磁臂换向，由外向里移动。这时，同样也是每次选择在当前磁道之内，且距离最近的进程来调度。这样磁头逐步地向里移动，直到再无更里面的磁道需要访问。显然，这种方式避免了饥饿现象的出现。在这种算法中，磁头移动的规律颇似电梯的运行，故又被称为电梯调度算法。

(4) 单向扫描调度算法 (CSCAN)。SCAN 存在这样的问题：当磁头刚从里向外移动过某一磁道时，恰有一进程请求访问此磁道，这时该进程必须等待，待磁头从里向外，再从外向里扫描完所有要访问的磁道后才处理该进程的请求，致使该进程的请求被严重地推迟。为了减少这种延迟，CSCAN 算法规定磁头只做单向移动。

### 3.6.5 文件管理

文件管理系统是操作系统中实现文件统一管理的一组软件和相关数据的集合，是专门负责管理和存取文件信息的软件机构，简称文件系统。文件系统的功能包括：按名存取，即用户可以按名存取，而不是按地址存取；统一的用户接口，在不同设备上提供同样的接口，方便用户操作和编程；并发访问和控制，在多道程序系统中支持对文件的并发访问和控制；安全性控制，在多用户系统中的不同用户对同一文件可有不同的访问权限；性能优化，采用相关技术提高系统对文件的存储效率、检索和读/写性能；差错恢复，能够验证文件的正确性，并具有一定的差错恢复能力。

#### 1. 文件的组织结构

文件的结构是指文件的组织形式。从用户角度看到的文件组织形式称为文件的逻辑结构，文件系统的用户只要知道所需文件的文件名就可以存取文件中的信息，而无须知道这些文件的实际存储位置。从实现的角度看，文件在文件存储器上的存放方式称为文件的物理结构。

##### 1) 文件的逻辑结构

文件的逻辑结构可分为两大类：一是有结构的记录式文件，它是由一个以上的记录构成的文件，故又称为记录式文件；二是无结构的流式文件，它是由一串顺序字符流构成的文件。

##### 2) 文件的物理结构

文件的物理结构是指文件的内部组织形式，即文件在物理存储设备上的存放方法。由于文件的物理结构决定了文件在存储设备上的存放位置，所以文件的逻辑块号到物理块号的转换也

是由文件的物理结构决定的。根据用户和系统管理上的需要,可采用多种方法来组织文件,常见的文件物理结构有连续结构、链接结构、索引结构和多个物理块的索引表。

需要说明的是,索引表是在文件创建时由系统自动建立的,并与文件一起存放在同一文件卷上。根据一个文件大小的不同,其索引表占用物理块的个数不等,一般占一个或几个物理块。多个物理块的索引表可以有两种组织方式:链接文件和多重索引方式。

UNIX 文件系统采用的是三级索引结构,在该文件系统中 inode 是基本的构件,它表示文件系统树状结构的结点。UNIX 文件索引表项分 4 种寻址方式:直接寻址、一级间接寻址、二级间接寻址和三级间接寻址。

## 2. 文件目录

为了实现按名存取,系统必须为每个文件设置用于描述和控制文件的数据结构,它至少应包括文件名和存放文件的物理地址,这个数据结构称为文件控制块(FCB),文件控制块的有序集合称为文件目录。换句话说,文件目录是由文件控制块组成的,专门用于文件的检索。文件控制块也称为文件的说明或文件目录项(简称目录项)。

文件目录结构的组织方式直接影响文件的存取速度,关系到文件的共享性和安全性,因此组织好文件的目录是设计文件系统的重要环节。常见的目录结构有 3 种:一级目录结构、二级目录结构和多级目录结构。

在采用多级目录结构的文件系统中,用户要访问一个文件,必须指出文件所在的路径名,路径名是从根目录开始到该文件的通路上所有各级目录名拼起来得到的。在各目录名之间、目录名与文件名之间需要用分隔符隔开。例如,在 MS-DOS 中分隔符为“\”,在 UNIX 中分隔符为“/”。绝对路径名(Absolute Path Name)是指从根目录“/”开始的完整文件名,即它是由从根目录开始的所有目录名以及文件名构成的。

## 3. 文件存储空间的管理

操作系统要将文件保存到外部存储器(简称外存或辅存)上,首先必须知道存储空间的使用情况,即哪些物理块是被“占用”的,哪些是“空闲”的。特别是大容量的磁盘存储空间被多用户共享时,用户执行程序经常要在磁盘上存储文件和删除文件,因此,文件系统必须对磁盘空间进行管理。外存空闲空间管理的数据结构通常称为磁盘分配表(Disk Allocation Table)。常用的空闲空间的管理方法有位示图、空闲区表、空闲块链和成组链接法 4 种。

(1) 位示图。这种方法是在外存上建立一张位示图(Bitmap),记录文件存储器的使用情况,如图 3-20 所示。每一位对应文件存储器上的一个物理块,取值 0 和 1 分别表示空闲和占用。例如,某文件存储器上位示图的大小为  $n$ ,物理块依次编号为 0, 1, 2, ...。假如计算机系统中字长为 32 位,那么在位示图中的第 0 个字(逻辑编号)对应文件存储器上的 0, 1, 2, ..., 31 号物理块;第 1 个字对应文件存储器上的 32, 33, 34, ..., 63 号物理块,以此类推。

|           |   |   |   |   |   |   |     |   |   |
|-----------|---|---|---|---|---|---|-----|---|---|
| 第0字       | 1 | 0 | 1 | 0 | 0 | 1 | ... | 0 | 1 |
| 第1字       | 0 | 1 | 1 | 1 | 0 | 0 | ... | 1 | 1 |
| 第2字       | 0 | 1 | 1 | 1 | 1 | 0 | ... | 1 | 0 |
| 第3字       | 1 | 1 | 1 | 1 | 0 | 0 | ... | 0 | 1 |
| ⋮         |   |   |   |   |   |   |     |   |   |
| 第 $n-1$ 字 | 0 | 1 | 0 | 1 | 0 | 1 | ... | 1 | 1 |

图 3-20 位示图示例

位示图法的主要特点是位示图的大小由磁盘空间的大小（物理块总数）决定，位示图的描述能力强，适合各种物理结构。

（2）空闲区表。将外存空间上的一个连续的未分配区域称为“空闲区”。操作系统为磁盘外存上的所有空闲区建立一张空闲表，每个表项对应一个空闲区，空闲表中包含序号、空闲区的第一块号、空闲块的块数和状态等信息。该方法适用于连续文件结构。

（3）空闲块链。每个空闲物理块中有指向下一个空闲物理块的指针，所有空闲物理块构成一个链表，链表的头指针放在文件存储器的特定位置上（如管理块中），不需要磁盘分配表，节省空间。每次申请空闲物理块只需根据链表的头指针取出第一个空闲物理块，根据第一个空闲物理块的指针可找到第二个空闲物理块，以此类推。

（4）成组链接法。UNIX 系统采用该方法。例如，在实现时系统将空闲块分成若干组，每 100 个空闲块为一组，每组的第一个空闲块登记了下一组空闲块的物理盘块号和空闲块总数。当某个组的第一个空闲块号等于 0，意味着该组是最后一组空闲块。

#### 4. 文件的共享和保护

##### 1) 文件的共享

文件共享是指不同用户进程使用同一文件，它不仅是不同用户完成同一任务所必需的功能，还可以节省大量的主存空间，减少由于文件复制而增加的访问外存的次数。文件共享有多种形式，采用文件名和文件说明分离的目录结构有利于实现文件共享。

常见的文件链接有硬链接和符号链接两种。

（1）硬链接，也称为基于索引结点的链接。采用硬链接可以将两个文件目录表目指向同一个索引结点的链接，从而达到文件共享的目的。

（2）符号链接。建立新的文件或目录，并与原来文件或目录的路径名进行映射。当访问一个符号链接时，系统通过该映射找到原文件的路径，再对其进行访问。其优点是可以跨越文件系统，甚至可以通过计算机网络连接到任何地方的计算机中的文件；缺点是其他用户读取符号链接的共享文件会比读取硬链接的共享文件增加读盘的次数。

##### 2) 文件的保护

文件系统对文件的保护常采用存取控制方式进行。所谓存取控制，就是不同的用户对文件的访问规定不同的权限，以防止文件被未经文件主同意的用户访问。存取控制方法包括以下几种：

（1）存取控制矩阵。理论上，存取控制方法可用存取控制矩阵，它是一个二维矩阵，其中一维列出计算机的全部用户，另一维列出系统中的全部文件，矩阵中的每个元素  $A_{ij}$  表示第  $i$  个用户对第  $j$  个文件的存取权限。通常，存取权限有可读 R、可写 W、可执行 X 以及它们的组合。存取控制矩阵在概念上是简单、清楚的，但在实现上却有困难。当一个系统用户数和文件数很大时，二维矩阵要占很大的存储空间，验证过程也将耗费许多系统时间。

（2）存取控制表。存取控制矩阵由于太大往往无法实现。一个改进的办法是按用户对文件的访问权限的差别对用户进行分类，由于某一文件往往只与少数几个用户有关，所以这种分类方法可使存取控制表大大简化。

(3) 用户权限表。改进存取控制矩阵的另一种方法是以用户或用户组为单位,将用户可存取的文件集中起来存入表中,这个表称为用户权限表。表中的每个表目表示该用户对对应文件的存取权限,这相当于存取控制矩阵一行的简化。

(4) 密码。在创建文件时,由用户提供一个密码,在文件存入磁盘时用该密码对文件内容加密。如果要对文件进行读取操作,需要将文件解密才能读取文件。

## 5. 文件系统的安全与可靠性

### 1) 系统的安全

系统的安全涉及两类不同的问题,一类涉及技术、管理、法律、道德和政治等问题,另一类涉及操作系统的安全机制。随着计算机应用范围的扩大,在所有稍具规模的系统中都会从多个级别来保证系统的安全性。一般从4个级别上对文件进行安全性管理:系统级、用户级、目录级和文件级。

### 2) 文件系统的可靠性

文件系统的可靠性是指系统抵抗和预防各种物理性破坏和人为性破坏的能力。比起计算机的损坏,文件系统的破坏往往后果更加严重,因为文件系统被破坏在很多情况下是无法恢复的。特别是对于程序文件、客户档案、市场计划或其他数据文件丢失的客户来说,这不亚于一场大的灾难。

保护信息的主要方法有转储和恢复、日志文件和文件系统的一致性检查。文件系统的一致性检查包括块的一致性检查和文件的一致性检查。

## 3.6.6 作业与用户界面

### 1. 基本概念

作业是系统为完成一个用户的计算任务(或一次事务处理)所做的工作总和。例如,对用户编写的源程序,需要经过编译、连接、装入及执行等步骤得到结果,这其中的每一个步骤称为作业步。在操作系统中用来控制作业进入、执行和撤销的一组程序称为作业管理程序。操作系统可以为每个作业创建作业步进程,完成用户的工作。

#### 1) 作业与作业控制方式

操作系统控制用户作业的运行采用脱机和联机控制两种方式。在脱机控制方式中,作业运行的过程是无须人工干预的,但用户必须将自己想让计算机干什么的意图用作业控制语言(JCL)编写成作业说明书,连同作业一起提交给计算机系统;在联机控制方式中,操作系统向用户提供了一组联机命令,用户可以通过终端输入命令将自己想让计算机干什么的意图告诉计算机,以控制作业的运行过程。

作业由程序、数据和作业说明书3个部分组成。作业说明书包括作业基本情况、作业控制、作业资源要求的描述,它体现用户的控制意图。其中,作业基本情况包括用户名、作业名、编程语言和最大处理时间等;作业控制描述包括作业控制方式、作业步的操作顺序、作业执行出错处理;作业资源要求描述包括处理时间、优先级、主存空间、外设类型和数量等。

## 2) 作业状态及转换

作业状态分为4种：提交、后备（也称收容）、执行和完成。其状态及转换如图3-21所示。

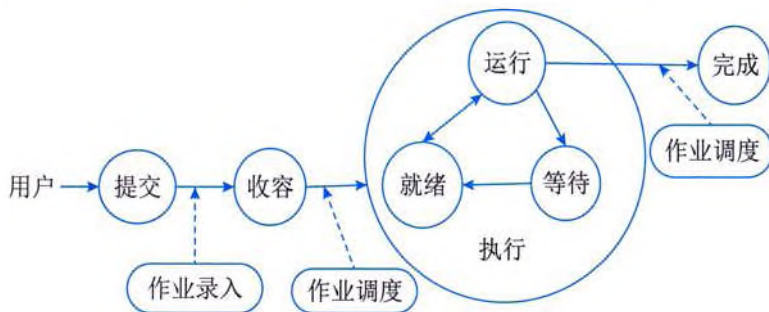


图 3-21 作业的状态及其转换

(1) 提交。作业在其处于输入设备进入外部存储设备的过程时的状态称为提交状态。

(2) 后备。通过 Spooling 系统将作业输入到计算机系统的后备存储器（磁盘）中，随时等待作业调度程序调度时的状态称为后备状态。

(3) 执行。一旦作业被作业调度程序选中，为其分配了必要的资源，并为其建立相应的进程后，该作业便进入了执行状态。

(4) 完成。当作业正常结束或异常终止时，作业进入完成状态。此时，由作业调度程序对该作业进行善后处理。如撤销作业的作业控制块，收回作业所占的系统资源，将作业的执行结果形成输出文件放到输出井中，由 Spooling 系统控制输出。

## 3) 作业控制块和作业后备队列

所谓作业控制块（JCB），是记录与该作业有关的各种信息的登记表。JCB 是作业存在的唯一标志，包括用户名、作业名和状态标志等信息。

由于在输入井中有较多的后备作业，为了便于作业调度，系统将 JCB 排成一个或多个队列，将其称为作业后备队列。换句话说，作业后备队列是由若干个 JCB 组成的。

## 2. 作业调度

在作业管理中，选择调度算法需要考虑三方面的因素：与系统的整个设计目标一致；均衡地使用系统资源；平衡系统和用户的要求。

### 1) 作业调度算法

常用的作业调度算法包括先来先服务、短作业优先、优先级调度算法和响应比高优先，前三种调度算法容易理解，本小节重点介绍响应比高优先算法。

响应比高优先算法是指响应比高的作业优先启动运行。响应比定义如下：

$$R_p = \frac{\text{作业响应时间}}{\text{作业执行时间}}$$

其中，作业响应时间为作业进入系统后的等待时间与作业的执行时间之和，即响应比也可记为：

$$R_p = 1 + \frac{\text{作业响应时间}}{\text{作业执行时间}}$$

对于响应比高优先算法,在每次调度前都要计算所有被选作业(在作业后备队列中)的响应比,然后选择响应比最高的作业执行。该算法比较复杂,系统开销大。

## 2) 作业调度算法性能的衡量指标

在一个以批量处理为主的系统中,通常用平均周转时间或平均带权周转时间来衡量调度性能的优劣。假设作业  $J_i (i=1,2,\dots,n)$  的提交时间为  $t_{si}$ , 执行时间为  $t_{ri}$ , 作业完成时间为  $t_{oi}$ , 则作业  $J_i$  的周转时间  $T_i$  和带权周转时间  $W_i$  分别定义为:

$$T_i = t_{oi} - t_{si} \quad (i=1,2,\dots,n), \quad W_i = T_i / t_{ri} \quad (i=1,2,\dots,n)$$

$n$  个作业的平均周转时间  $T$  和平均带权周转时间  $W$  分别定义为:

$$T = \frac{1}{n} \sum_{i=1}^n T_i, \quad W = \frac{1}{n} \sum_{i=1}^n W_i$$

从用户的角度来说,总是希望自己的作业在提交后能立即执行,这意味着当等待时间为 0 时作业的周转时间最短,即  $T_i = t_{ri}$ 。但是,作业的执行时间  $t_{ri}$  并不能直观地衡量系统的性能,而带权周转时间  $W_i$  却能直观地反映系统的调度性能。从整个系统的角度来说,不可能满足每个用户的这种要求,而只能使系统的平均周转时间或平均带权周转时间最小。

## 3. 用户界面

用户界面 (User Interface) 是计算机中实现用户与计算机通信的软/硬件部分的总称。用户界面也称用户接口或人机界面。操作系统提供命令接口、程序接口和图形界面接口。通常,命令接口和图形界面接口是面向用户和管理者的;程序接口(如系统调用)是面向程序开发人员的。

用户界面的硬件部分包括用户向计算机输入数据或命令的输入装置,以及由计算机输出供用户观察或处理的输出装置。用户界面的软件部分包括用户与计算机相互通信的协议、约定、操纵命令及其处理软件。目前,常用的输入/输出装置有键盘、鼠标、显示器和打印机等。常用的人机通信方法有命令语言、选项、表格填充及直接操纵等。

从计算机用户界面发展过程来看,用户界面可分为控制面板式用户界面、字符用户界面、图形用户界面和新一代用户界面 4 个阶段。

虚拟现实技术将用户界面的发展推向一个新阶段:人将作为参与者,以自然的方式与计算机生成的虚拟环境进行通信。以用户为中心、自然、高效、高带宽、非精确、无地点限制等是新一代用户界面的特征。多媒体、多通道及智能化是新一代用户界面的技术支持。语音、自然语言、手势、头部跟踪、表情和视线跟踪等新的、更加自然的交互技术将为用户提供更方便的输入技术。计算机将通过多种感知通道来理解用户的意图,实现用户的要求。计算机不仅以二维屏幕向用户输出,而且以真实感(立体视觉、听觉、嗅觉和触觉等)的计算机仿真环境向用户提供真实的体验。

### 3.6.7 国产操作系统

国产操作系统主要以开源的 Linux 为基础进行二次开发，目前认知度较高的国产操作系统主要包括银河麒麟、深度、统信、中标麒麟、红旗 Linux、安超 OS、中科方德和 StartOS 等。

(1) 银河麒麟操作系统 (Kylin OS) 是由国防科技大学、中国软件与技术服务股份有限公司、联想集团、浪潮集团有限公司和民族恒星公司合作研制的。其目标是打破国外操作系统的垄断，研发一套中国自主知识产权的服务器操作系统。该系统完全版共包括三个版本：实时版、安全版、服务器版。

(2) 深度操作系统 (deepin) 是基于 Linux 内核，以桌面应用为主的开源 GNU/Linux 操作系统，支持笔记本、台式机和一体机。该系统包含深度桌面环境 (DDE) 和近 30 款深度原创应用，以及数款来自开源社区的应用软件，支撑广大用户日常的学习和工作。

(3) 统信操作系统 (UOS) 是由统信软件技术有限公司研发的，基于 deepin 进行深度开发，目前已是一款较为完善的系统。

(4) 中标麒麟操作系统是由上海中标软件有限公司发布的面向桌面应用的操作系统。采用强化的 Linux 内核，分桌面版、通用版、高级版和安全版等。

(5) 红旗 Linux 是由北京中科红旗软件技术有限公司开发的一系列 Linux 发行版，包括桌面版、工作站版、数据中心服务器版、HA 集群版和红旗 Linux 嵌入式版。红旗 Linux 是国内较大、较成熟的 Linux 发行版，也是比较出名的国产操作系统之一。

(6) 安超 OS 2020 是一套基于服务器架构的通用型云操作系统，具有软硬件解耦、应用优化、支持混合业务负载等特点。为企业提供高性能、高可用、高效率且易于安装维护的 IT 基础设施平台，加速政府和企业上云进程，为推动企业数字化转型提供完整的一站式企业上云的云操作系统平台和生态解决方案。

(7) 中科方德基于核高基（核心电子器件、高端通用芯片及基础软件产品）桌面操作系统基础版，采用核高基安全加固内核，与基于国产兆芯处理器的整机进行全面适配和深度优化，安装简单，易配置。

(8) StartOS（起点操作系统）的前身是由广东雨林木风计算机科技有限公司的 ylmfos 开发组所研发的 ylmfos。该系统是从 Linux 底层构建的，拥有完全自主的核心配置及特色。具有全新的包管理、全新的操作界面，是一个非常符合中国人操作习惯的桌面 Linux 操作系统。